

Vue

作者： -- 天气预报

日期： -- 2023-10-12

Vue

作者： -- 天气预报

日期： -- 2023-10-12

01. 基础

1.1 参考指南

1.2 项目构建

1.3 运行

1.4 目录清理

02. 单文件组件

2.1 逻辑概念

2.2 插值表达式

2.3 响应式

2.4 计算属性

2.5 响应式监听

2.6 指令

2.7 事件修饰符

2.8 静态资源

03. 组件高级

3.1 组件引用与注册

3.2 组件传值

3.3 动态组件

3.4 内置组件

- 3.5 组件插槽
- 3.6 组件生命周期
- 04. Axios
 - 4.1 安装
 - 4.2 GET/POST
 - 4.3 封装
- 05. Vue Router
 - 5.1 路由概念机制
 - 5.2 安装
 - 5.3 路由导航
 - 5.4 路由组件及子路由
 - 5.5 动态路由
 - 5.6 编程式路由
 - 5.7 路由守卫
- 06. Pinia
 - 6.1 状态管理概念
 - 6.2 安装
 - 6.3 模块定义
 - 6.4 状态使用
 - 6.5 持久化
- 07. 组合式api
- 08. 开源组件
 - 8.1 Paginate
 - 8.2 VeeValidate
- 09. 友情链接

01. 基础

≡ 必要前提技术：ES6、Node、Npm、Sass [Webpack | Vite]

1.1 参考指南

v2 文档: <https://v2.cn.vuejs.org/>

v3 文档: <https://cn.vuejs.org/>

- v2 的终止支持时间是 2023-12-31 (本教程只会从零介绍 v3 技术栈)



1.2 项目构建

≡ vue@latest 是官方提供创建 v3 项目的模板、基于 Vite 构建项目

- npm



```
npm create vue@latest
```

- 根据脚手架引导安装依赖及环境



```
✓ Project name: ... <your-project-name>
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit testing? ... No / Yes
✓ Add an End-to-End Testing Solution? ... No / Cypress / Playwright
✓ Add ESLint for code quality? ... No / Yes
✓ Add Prettier for code formatting? ... No / Yes
```

```
Scaffolding project in ./<your-project-name>...
Done.
```

- 项目运行



```
> cd <your-project-name>

> npm install
> npm run dev
```

1.3 运行

- 创建项目

Vue.js - The Progressive JavaScript Framework

```
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit Testing? ... No / Yes
✓ Add an End-to-End Testing Solution? » No
✓ Add ESLint for code quality? ... No / Yes
```

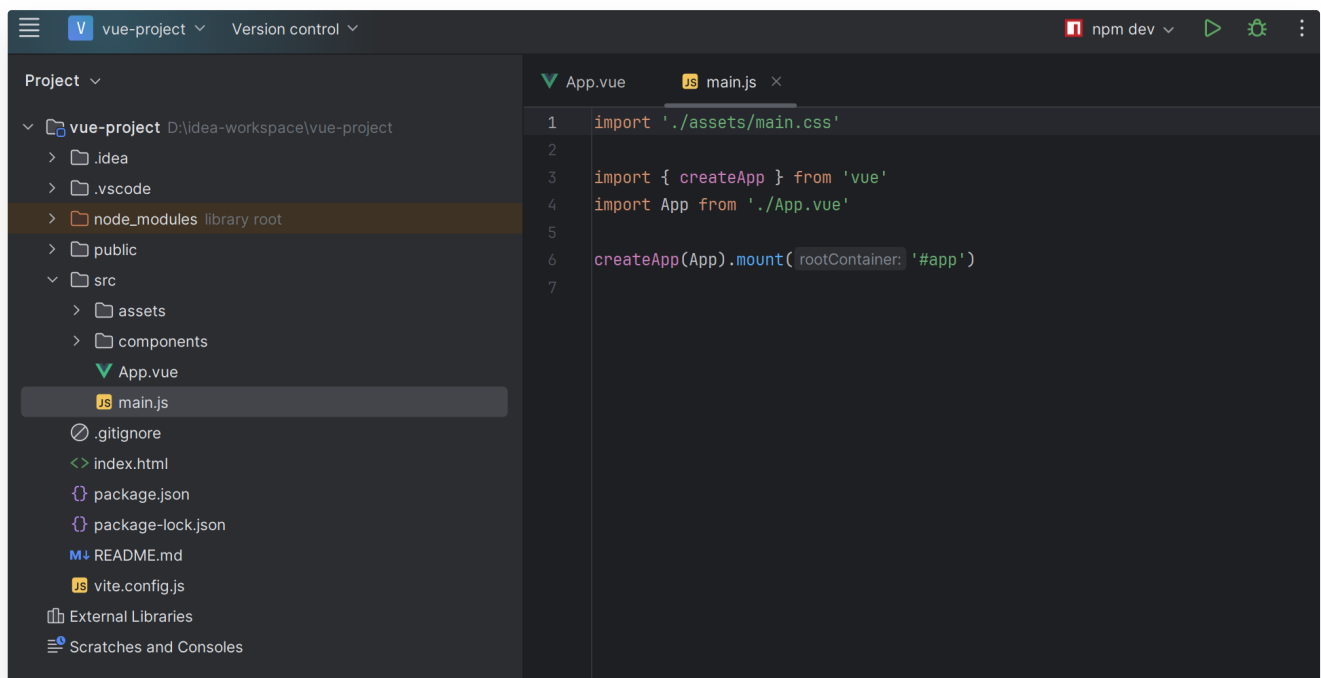
Scaffolding project in C:\Users\xbinu\AppData\Local\Temp\1696993389173-0\vue-project...

Done. Now run:

```
cd vue-project
npm install
npm run dev
```

Done

- npm install (安装依赖)

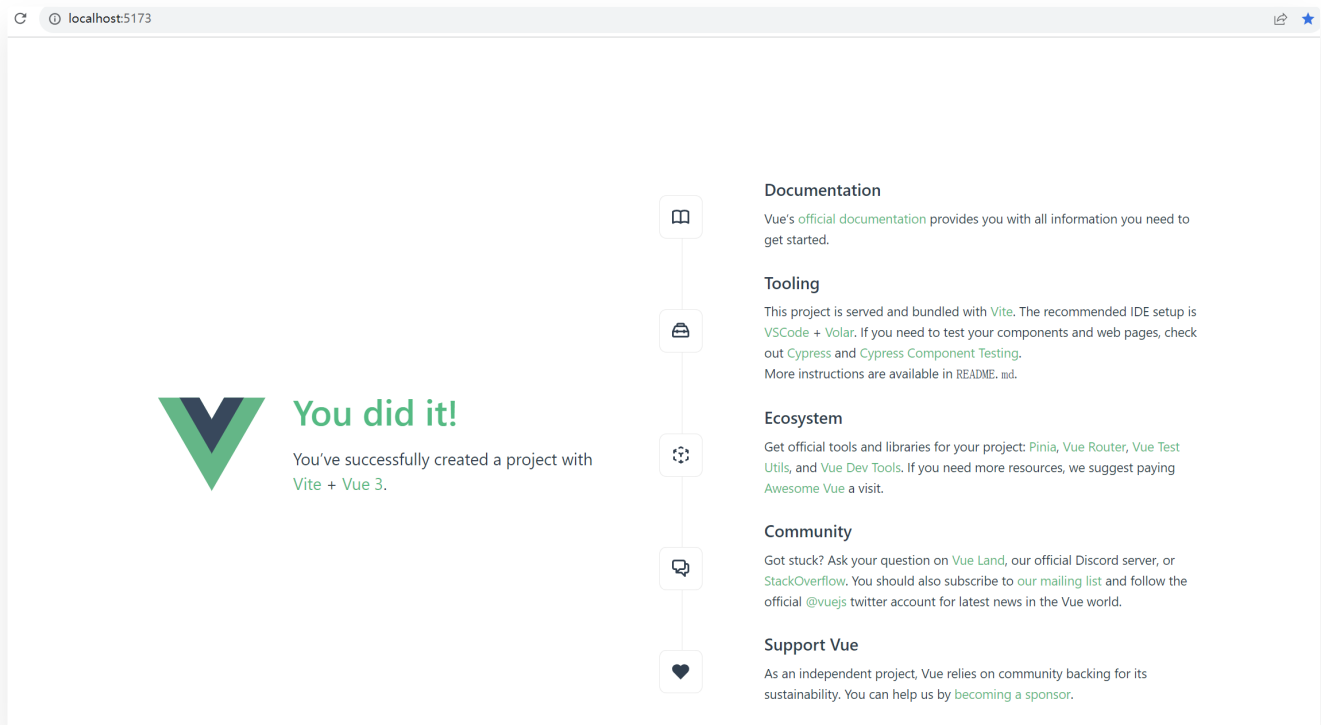


目录说明

- ① main.js 是 vite & vue 环境运行的 入口 javascript 文件
- ② App.vue 是 vue 渲染根组件、表示内容从这里开始渲染到 index.html 模版

- ③ `index.html` 是渲染的模板 HTML 空文件、里面没有内容
- ④ `assets` 约定的静态资源文件目录 `css/img/font` 等
- ⑤ `components` 约定的组件文件目录

- `npm run dev` (<http://localhost:5173/> 下图表示运行成功)



1.4 目录清理

- 删除 `/assets/base.css`
- 删除 `/assets/main.css`
- 删除 `/components/`全部示例组件文件
- `main.js`



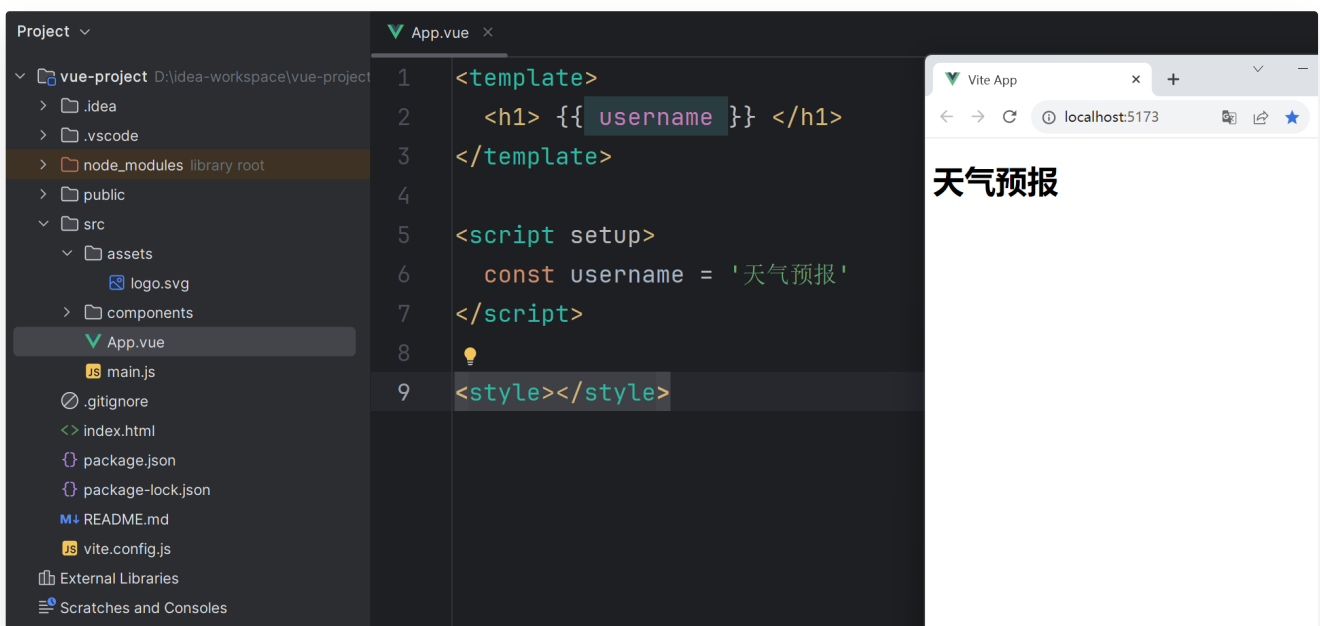
```
import {createapp} from 'vue'  
import app from './app.vue'  
  
const app = createapp(app)  
  
app.mount('#app')
```

- **app.vue**



```
<template>  
  <h1>Vue3</h1>  
</template>  
  
<script setup></script>  
  
<style></style>
```

- 如图



02. 单文件组件

≡ * .vue 是单文件组件、由 `<template/>`、`<script>`、`<style>` 构成。

2.1 逻辑概念

- ① 单文件组件逻辑含义为 `template` 编写当前一小段 HTML 片段作为内容呈现
- ② `script` 完成当前 HTML 片段的 javascript 交互逻辑
- ③ `style` 完成当前 HTML 片段的 css 样式
- ④ 一个 .vue 单文件就是一块功能完整的 HTML 区域、组件可以像被 HTML 标签引用
- ⑤ 如 `<Nav>` 、`<Banner>`、`<Footer>`、`<Main>`、`<LoginForm>` ...
- ⑥ 将多个单文件组件组装即可得到 功能完整的页面
- ⑦ 显然：单文件组件有利于功能开发、维护、有利于业务处理封装、复用等

2.2 插值表达式

≡ `{{ 表达式 }}` 用于定义表达式 或 引用 `<script>` 中声明的变量

- `{{ }}` 只能是表达式、不能是语句 `}}` 如 `{{ let message = '这是语句' }}`
- `{{ }}` 简单理解为单行代码且有返回值的就是表达式 `}}`
- `app.vue`



```
<template>
  <h2> vue 插值表达式 </h2>
  <h2>{{ 'vue 插值表达式' }}</h2>
  <h2>{{ 8 > 5 ? 'true' : 'false' }}</h2>
  <h2>{{ username }}</h2>
  <h2>username : {{ username }}</h2>
  <h2>{{ message() }}</h2>
  <h2>{{ html }}</h2>
</template>

<script setup>
  const username = '天气预报'

  const message = () => '这是一个箭头函数'

  const html = '<i>i 元素不会被渲染</i>'
</script>
```

2.3 响应式

≡ 当插值表达式内容发生变化时、期望 `<template>` HTML 重新渲染则为响应式。

01. ref

- `ref(初始值)` 函数包装的变量为响应式变量、该值变更时会触发重新渲染
- `ref()` 函数需从 `vue` 模块导入: `import {ref} from 'vue'`
- **app.vue [操作变量]**



```
<template>
  <h1>uuid : {{ uuidRef }}</h1>
</template>

<script setup>
```

```
// npm i uuid
// uuid 是一款 uuid 库 v4 是一个 uuid 函数()

import {v4 as uuid} from 'uuid'
import {ref} from 'vue'

const uuidRef = ref(uuid())
setInterval(() => uuidRef.value = uuid(), 2000)

</script>
```

- app.vue [操作对象]



```
<template>
  <h1>id: {{ userRef.id }}</h1>
  <h1>username: {{ userRef.username }}</h1>
  <h1>gender: {{ userRef.gender }}</h1>
</template>

<script setup>

  import {v4 as uuid} from 'uuid'
  import {ref} from 'vue'

  const userRef = ref({
    id: uuid(),
    username: 'lucas',
    gender: 'gender'
  })

  setInterval(() => userRef.value.id = uuid(), 2000)

</script>
```

- app.vue [操作数组]



```
<template>
  <h1>array: {{ arrayRef }}</h1>
</template>

<script setup>
  import {ref} from 'vue'

  const arrayRef = ref([1, 2])

  setInterval(() => arrayRef.value.push(Math.random()), 3000)
</script>
```

总结

- ① 响应式数据需要使用 `ref`(初始值) 函数进行包装
- ② 操作 `ref` 响应式数据、需以 `.value` 形式才能访问到具体的值
- ③ 推荐响应式变量 命名为 `xxxRef` 同时 `.value` 也可使得敏感响应式变量

注意事项

≡ `xxRef.value` 具体值可以进行解构、但解构出的值为只读值、且丢失响应式

- `app.vue` [解构操作]



```
<template>
  <h1>id: {{ userRef.id }}</h1>
  <h1>username: {{ userRef.username }}</h1>
  <h1>gender: {{ userRef.gender }}</h1>

  <h2>此处 id 不会变化 {{ id }}</h2>
  <h2>{{ username }}</h2>
  <h2>{{ gender }}</h2>
</template>

<script setup>
```

```

import {v4 as uuid} from 'uuid'
import {ref} from 'vue'

const userRef = ref({
  id: uuid(),
  username: 'lucas',
  gender: 'gender'
})

setInterval(() => userRef.value.id = uuid(), 2000)

const {id, username, gender} = userRef.value
</script>

```

02. reactive

- reactive(引用类型) 同理 ref 仅能操作对象、数组等引用类型
- reactive(初始值) 返回的引用可直接操作、无需 .value
- reactive({}) 解构操作依然为只读、且丢失响应式

- app.vue

```

● ● ●

<template>
  <h1>{{ user }}</h1>

  <h1>{{ id }}</h1>
  <h1>{{ username }}</h1>
  <h1>{{ gender }}</h1>
</template>

<script setup>
  import {v4 as uuid} from 'uuid'
  import {reactive} from 'vue'

  const user = reactive({
    id: uuid(),

```



```

      username: 'lucas',
      gender: 'gender'
    })

    setInterval(() => user.id = uuid(), 2000)

    const { id, username, gender } = user
  </script>

```

03. toRef

- toRef(**响应式对象**, '属性名称') 取出响应式对象中的某个属性、且保持同源
- 响应式对象、取出的响应式属性会联动更新

- app.vue

```

● ● ●

<template>
  <h1>{{ user }}</h1>
  <h1>toRefId : {{ idRef }}</h1>
</template>

<script setup>
  import {v4 as uuid} from 'uuid'
  import {reactive, toRef} from 'vue'

  const user = reactive({
    id: uuid(),
    username: 'lucas',
    gender: 'gender'
  })

  setInterval(() => user.id = uuid(), 2000)

  const idRef = toRef(user, 'id')
</script>

```

04. toRefs

- toRefs(响应式对象) 将参数转为普通对象、但该对象的每个属性指向响应式对象属性
- 响应式对象会影响普通对象、反之这不行

- app.vue

```
● ● ●

<template>
  <h1>{{ user }}</h1>
  <h1>ordinaryUser {{ ordinaryUser }}</h1>
</template>

<script setup>

  import {v4 as uuid} from 'uuid'
  import {reactive, toRefs, toRef} from 'vue'

  const user = reactive({
    id: uuid(),
    username: 'lucas',
    gender: 'gender'
  })

  const ordinaryUser = toRefs(user)

  setInterval(() => {
    user.id = uuid()
    ordinaryUser.username = toRef('jerry')
  }, 2000
)

</script>
```

05. readonly

- `readonly`(普通对象或变量|响应式|ref) 返回原值的只读代理

- `app.vue`

```
● ● ●

<template>
  <h1>{{ user }}</h1>
  <h1>{{ readonlyUser }}</h1>
</template>

<script setup>
  import {v4 as uuid} from 'uuid'
  import {reactive, readonly} from 'vue'

  const user = reactive({
    id: uuid(),
    username: 'lucas',
    gender: 'gender'
  })

  const readonlyUser = readonly(user)

  user.username = 'jimmy'

  // 错误 readonlyUser 只读
  // readonlyUser.username = 'jerry'
</script>
```

06. isRef

- `isRef`(变量) 判断参数变量是否为 `ref`
- `isReactive`(变量) 判断参数变量是否为 `reactive`

- `app.vue`



```
<template>
  <h1>{{ number }}</h1>
  <h1>{{ user }}</h1>

  <h1>{{ isRef1 }}</h1>
  <h1>{{ isRef2 }}</h1>
</template>

<script setup>
  import {isRef, ref, isReactive, reactive} from 'vue'

  const number = ref(Math.random())

  const user = reactive({
    username: 'lucas',
    gender: 'male'
  })

  const isRef1 = isRef(number)
  const isRef2 = isReactive(user)
</script>
```

07. isReadonly

- 检查传入的值是否为只读对象
- 注意：不能通过传入的对象直接赋值、只读对象的属性也不能更改
- **app.vue**



```
<template>
  <h1>{{ 'isReadonly' }}</h1>
</template>

<script setup>
  import {readonly, isReadonly} from 'vue'

  const user = readonly({
```

```

        username: 'lucas',
        gender: 'male'
      }
    )

    console.log(isReadonly(user))

    // 错误 vue readonly 返回对象的引用、及属性都不可更改

    user.username = 'jimmy'
    console.log(user)

</script>

```

08. unRef

- 返回 unRef(参数是普值返回本身 | 参数是 ref 返回 ref.value)

- app.vue

```

● ● ●

<template>
  <h1>{{ 'unRef' }}</h1>
</template>

<script setup>
  import {ref, unref} from 'vue'

  const numberRef = ref(100)

  // const value = isRef(numberRef) ? numberRef.value : numberRef

  const value = unref(numberRef)
  console.log(value)
</script>

```

09. shallowRef

- 与 ref 不同、shallowRef 是浅层响应式、只对 .value 响应式
 - `const user = shallowRef({name:'lucas'})`
 - `user.value = {}` 是响应式、`user.value.name = ''` 不是响应式
-
- `app.vue`

```
● ● ●

<template>
  <h1>{{ 'shallowRef' }}</h1>
  <h1>{{ userShallowRef }}</h1>
</template>

<script setup>
  import {shallowRef} from 'vue'

  const userShallowRef = shallowRef({
    username: 'lucas',
    gender: 'male'
  })

  setTimeout(() => {
    userShallowRef.value =
      {...userShallowRef.value, username: 'jimmy'}

    }, 2000)

  setTimeout(() => {
    userShallowRef.value.gender = 'female'
    }, 3000)

</script>
```

2.4 计算属性

≡ 计算属性用于缓存计算结果、且完成计算逻辑封装[通常是计算响应式数据]

- computed(有返回值的函数) computed 会返回一个只读的 ref 引用
- computed 返回的值被 vue 称为计算属性
- 需要注意的是：计算属性是带有响应式数据源追踪的

- app.vue

```

● ● ●
<template>
  <h1>{{ counter }}</h1>
  <h2>计算属性：{{ doubleCounter }}</h2>
  <h2>计算属性：{{ doubleCounter }}</h2>
  <h2>计算属性：{{ doubleCounter }}</h2>
</template>

<script setup>

  import {ref, computed} from 'vue'

  const counter = ref(100)

  const doubleCounter = computed(() => {
    counter.value = counter.value * 2
    console.log('counter 不变时、doubleCounter 始终不会重新执行')
    return counter
  })

</script>
```

2.5 响应式监听

≡ 如果期望对响应式变量值发生改变时、作出逻辑操作、即可监听响应式变量

01. watch

- 侦听 **当前组件** 一个或多个响应式数据源、并在数据源变化时调用所给的回调函数
- 执行次数：0 ~ N
- 注意 watch 也可在子组件中 监听父组件传递过来的响应式

- app.vue

```

● ● ●
<template>
  <h1>{{ number }}</h1>
</template>

<script setup>

  import {ref, watch} from 'vue'

  const number = ref(Math.random())

  setInterval(() => number.value = Math.random(), 3000)

  watch(number, (_new, _old) => {

    // _new 当前新值、_old 更新前的值
    console.log(_new, _old)
  })

</script>
```

- 子组件监听父组件


```
App.vue
1 <template>
2   <h1>父组件: {{ number }}</h1>
3   <Child :number="number"></Child>
4 </template>
5
6 <script setup>
7
8 import {ref} from 'vue'
9 import Child from '@/components/Child.vue'
10
11 const number = ref(Math.random())
12
13 setInterval( handler: () => number.value = Math.random(), timeout: 2000)
14
15
16 </script>

Child.vue
1 <template>
2   <h1>子组件: {{ number }}</h1>
3 </template>
4
5 <script setup>
6
7 import {watch} from "vue";
8
9 const props = defineProps( props: {
10   number: Number
11 })
12
13 watch( source: () => props.number, cb: (_new, _old) => {
14   console.log(_new, _old, '子组件...')
15 })
16
17 </script>
18
```

- 此处注意 watch 参数问题



正确的 监听 props
watch(props, 回调)

正确的 监听 props.fileName
watch(() => props.fileName, 回调)

错误的
watch(props.fileName, 回调)

错误的
watch(() => props, 回调)

02. watchEffect

- 立即运行 watchEffect 参数函数且响应式地追踪其依赖、并在依赖更新时再次执行
- watchEffect 执行次数: 1 ~ N
- watchEffect 内部若操作响应式数据源且变更时、还会触发执行

- app.vue



```
<template>
  <h1>父组件: {{ number }}</h1>
</template>

<script setup>

  import {ref, watchEffect} from 'vue'

  const number = ref(Math.random())

  setInterval(() => number.value = Math.random(), 2000)

  watchEffect(() => {

    // 请注释该行代码进行测试观察
    console.log(number.value)
    console.log('...')
  })

</script>
```

2.6 指令

≡ 指令是 Vue 特有且针对 template HTML 渲染行为的预定属性

01. v-text

≡ v-text 绑定文本、绑定 HTML 元素不能有文本内容

- app.vue

```
● ● ●  
  
<template>  
  <h1>number: {{ number }}</h1>  
  <h1 v-text="number"></h1>  
</template>  
  
<script setup>  
  const number = 100  
</script>
```

02. v-html

≡ v-html 绑定文本、内容中 HTML 元素会被渲染

- app.vue

```
● ● ●  
  
<template>  
  <h1 v-text="content"></h1>  
  <h1 v-html="content"></h1>  
</template>  
  
<script setup>  
  const content = '<i>HTML I 元素</i>'  
</script>
```

03. v-show

≡ v-show 以 css display: none 方式控制元素是否显式

- app.vue

```
● ● ●  
  
<template>  
  <h1 v-show="showRef">显式元素</h1>  
</template>  
  
<script setup>  
  import {ref} from 'vue'  
  
  const showRef = ref(true)  
  
  setInterval(() => showRef.value = !showRef.value, 2000)  
</script>
```

04. v-if

≡ v-if 以 dom 移除/添加 方式控制元素是否显式

- v-if 可独立使用、逻辑为真则显式元素、否则不显式
- v-if / 还可配合 v-else 一起使用、但 v-else 必须 **紧贴上一个 v-if**

- app.vue



```

<template>
  <h1 v-if="showRef">显式元素1</h1>
  <h1 v-else>显式元素2</h1>
</template>

<script setup>
  import {ref} from 'vue'

  const showRef = ref(true)

  setInterval(() => showRef.value = !showRef.value, 2000)
</script>

```

05. v-for

≡ v-for 遍历生成 HTML 结构、v-for 写在刚好每次都要生成结构的主元素上

- :key 是 vue 内部属性 用于为每次循序结构绑定唯一值、避免重复渲染
- app.vue



```

<template>
  <div>
    <ul>
      <li v-for="(e, i) in array" :key="e">{{ e }}</li>
    </ul>

    <ul>
      <li v-for="number in 10" :key="number">{{ number }}</li>
    </ul>
  </div>
</template>

<script setup>
  const array = ['a', 'b', 'c', 'd']

```

```
</script>
```

06. v-bind

≡ v-bind 用于绑定 HTML 元素属性、自定义属性也可

- v-bind:属性名称="变量值" 也可简写为 :属性名称="变量值"
- app.vue

```
● ● ●  
<template>  
  
  <div>  
    <h1><a v-bind:href="url">淘宝1</a></h1>  
    <h1><a :href="url">淘宝2</a></h1>  
  </div>  
  
</template>  
  
<script setup>  
  const url = 'https://www.taobao.com'  
</script>
```

07. v-on

≡ v-on 用于绑定 javascript 原生事件、或自定义事件

- v-on:事件名称="处理函数" 也可简写为 @事件名称.修饰符="处理函数"

- app.vue



```
<template>
  <div>
    <h1>counter : {{ counter }}</h1>
    <h1>
      <button v-on:click="incr">按钮1</button>
    </h1>
    <h1>
      <button @click="incr">按钮2</button>
    </h1>

    <!-- 不要写成下面这样 incr 是传递的函数对象参数、incr() 调用 incr 函数 -->
    <!-- <h1><button @click="incr()">按钮2</button></h1>-->
  </div>
</template>
<script setup>
  import {ref} from 'vue'

  const counter = ref(1)

  function incr() {
    counter.value++
  }
</script>
```

08. v-model

≡ v-model 针对 input/select/textarea 的双向绑定指令

- v-model 绑定 input、textarea [input事件]
- v-model 绑定 select [change事件]

- app.vue



```

<template>

  <div>
    <h1>usernameRef: {{ usernameRef }}</h1>
    <input type="text" v-model="usernameRef">

    <h1>contentRef:{{ contentRef }}</h1>
    <textarea v-model="contentRef"></textarea>

    <h1>optionRef : {{ optionRef}}</h1>
    <select name="" id="" v-model="genderRef">
      <option value="male">男</option>
      <option value="female">女</option>
    </select>
  </div>

</template>

<script setup>
  import {ref} from 'vue'

  const usernameRef = ref('')
  const contentRef = ref('')
  const genderRef = ref('male')
</script>

```

09. v-pre

≡ v-pre 作用元素内部的 {{}} 不会被渲染

- app.vue



```
<template>
  <h1 v-pre>{{ usernameRef }}</h1>
</template>

<script setup>
  import {ref} from 'vue'
  const usernameRef = ref('lucas')
</script>
```

10. 第三方指令

≡ @formkit/auto-animate 可实现子元素变动动画[删除、添加、移动]

- npm



```
npm i @formkit/auto-animate
```

- main.js [挂载]



```
import {createApp} from 'vue'
import App from './App.vue'

import { autoAnimatePlugin } from '@formkit/auto-animate/vue'

const app = createApp(App)

app.use(autoAnimatePlugin)
app.mount('#app')
```

- app.vue

```

● ● ●

<script setup>
import {ref} from 'vue'

const items = ref(["😄", "😐", "😐", "😐", "😐"])

function removeItem(toRemove) {
  items.value = items.value.filter((item) => item !== toRemove)
}
</script>

<template>
  <h5>Click emojis to remove them.</h5>
  <ul v-auto-animate>
    <li
      v-for="item in items"
      :key="item"
      @click="removeItem(item)">
      {{ item }}
    </li>
  </ul>
</template>
```

2.7 事件修饰符

≡ 事件修饰符是 vue 对事件对象轻微处理、通过预制修饰符处理 event 对象

- 部分修饰符可组合使用、但多个修饰符有时可能需注意使用顺序
- 使用方式 <a @click.prevent="handleClick"> 表示阻止超链接默认行为

01. 点击事件修饰符

- `.stop`
- `.prevent`
- `.self`
- `.capture`
- `.once`
- `.passive`

02. 按键修饰符

- `.enter`
- `.tab`
- `.delete`
- `.esc`
- `.space`
- `.up`
- `.down`
- `.left`
- `.right`

03. 鼠标按键修饰符

- `.left`
- `.right`
- `.middle`

04. 输入框修饰符

- `.lazy`
- `.number`
- `.trim`

- `app.vue`

```

● ● ●
<template>

  <div>
    <h1>
      <span>.prevent 阻止默认行为</span>
      <a href="https://www.taobao.com"
        @click.prevent="() => console.log('...')">淘宝</a>

    </h1>

    <p>.enter 修饰符 仅键盘回车键触发</p>
    <p>keyword : {{ keywordRef }}</p>
    <input type="search"
      v-model="keywordRef"
      @keyup.enter="() => console.log(keywordRef)">

  </div>

</template>

<script setup>
  import {ref} from 'vue'
  const keywordRef = ref('')
</script>
```

2.8 静态资源

≡ vue 预留 src/assets | src/public 目录存放静态资源

01. 图片引用

- template 中直接引用 public 目录资源使用 绝对路径 /
- template 中直接引用 assets 目录资源使用 相对路径 ./ 或 ../

- app.vue

```
  
<template>  
    
</template>
```

02. 样式绑定

≡ <style> 用于编写 CSS 样式、scoped 属性表示当前样式仅在本组件生效

- 未声明 scoped 约束的样式将会在混合使用组件时、影响到其它组件样式行为

- app.vue

```

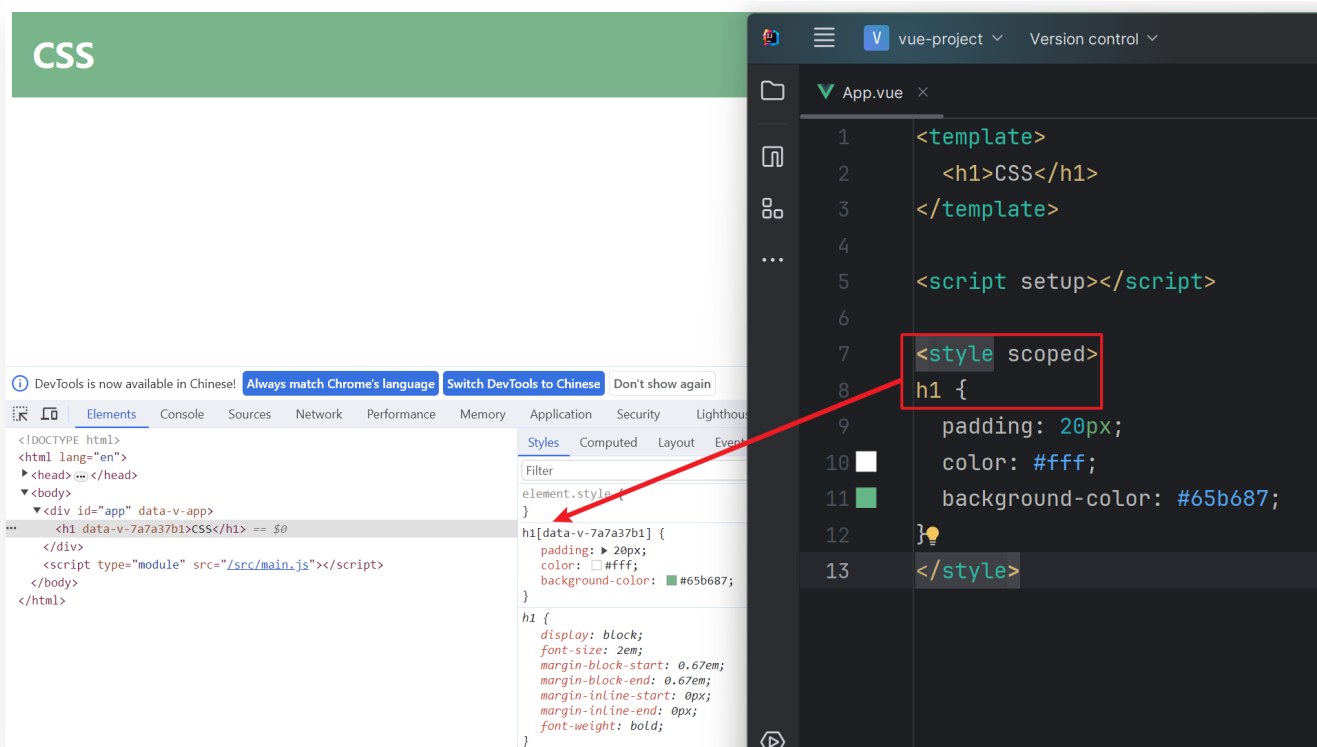
<template>
  <h1>CSS</h1>
</template>

<script setup></script>

<style scoped>
h1 {
  padding: 20px;
  color: #fff;
  background-color: #65b687;
}
</style>

```

- 如图 编写的 CSS 样式 会被 Vue 属性化处理



03. 外部样式表

≡ 外部样式表可在 `main.js` 中导入表示全局生效、会影响所有使用的组件

- 外部样式表也可在某个组件中导入、表示仅在该组件生效
- 推荐组件局部样式仅在该组件导入、UI框架通常在多个组件中使用、进行全局导入

- `src/assets/styles.css`

```
h1 {  
  padding: 20px;  
  color: #fff;  
  background-color: #65b687;  
}
```

- `main.js` [全局]

```
import {createapp} from 'vue'  
import app from './app.vue'  
  
import './assets/styles.css'  
  
createapp(app).mount('#app')
```

- `app.vue` [组件导入]



```
<template>
  <h1>CSS</h1>
</template>

<script setup>
  import './assets/styles.css'
</script>
```

04. Sass

≡ 使用 SASS 需安装 SASS 编译器: `npm install sass`

- sass 完成后、`<style scoped lang="scss">` 指定 sass|scss 即可
- `app.vue`



```
<template>
  <section>
    <h1>sass</h1>
    <p>vue & sass 安装与使用</p>
  </section>
</template>

<style scoped lang="scss">
  section {
    background-color: #eee;
    padding: 2rem;

    h1 {
      font-size: 36px;
    }

    p {
      font-size: 20px;
      font-weight: bold;
      color: #65b687;
    }
  }
}
```



```
}  
  
}  
</style>
```

05. Bootstrap

≡ 官网: <https://v5.bootcss.com/> [当前最新版 5.3.z]

- BOOTSTRAP 是一款结构型 CSS 响应式 UI 框架

 中文文档 示例 图标库 精选模板 官方博客

🔄 🐦 🌐 v5.3 🌞

v5.3 版本新增 Color mode support, expanded color palette, and more!



用 Bootstrap 构建快速、响应式布局的网站

强大、可扩展、功能丰富的前端开发工具包。它提供的功能包括：基于 Sass 进行编译和定制、内置的网格系统和组件、强大的 JavaScript 插件等。

```
$ npm i bootstrap@5.3.0-alpha1
```

📄 进入中文文档

当前版本 v5.3.0-alpha1 · [下载](#) · [v4.6.x 中文文档](#) · [所有版本列表](#)

<>

≡ 安装 `npm i bootstrap sass`

- main.js [全局导入]



```
import {createapp} from 'vue'
import app from './app.vue'

import 'bootstrap/scss/bootstrap.scss'
import 'bootstrap/dist/js/bootstrap.bundle.js'

createapp(app).mount('#app')
```

- app.vue



```
<template>
  <header class="navbar navbar-expand-lg navbar-dark bg-primary">
    <nav class="container">

      <a class="navbar-brand fw-bold" href="#">Bootstrap</a>

      <button class="navbar-toggler rounded-0"
        data-bs-toggle="collapse"
        data-bs-target="#collapse">
        <span class="navbar-toggler-icon" />
      </button>

      <div class="navbar-collapse collapse" id="collapse">

        <ul class="navbar-nav me-auto">

          <li class="nav-item">
            <a class="nav-link" href="#">vite</a>
          </li>

          <li class="nav-item">
            <a class="nav-link" href="#">vue</a>
          </li>

          <li class="nav-item">
            <a class="nav-link" href="#">vue router</a>
          </li>

        </ul>
      </div>
    </nav>
  </header>
</template>
```

```

<li class="nav-item">
  <a class="nav-link" href="#">pinia</a>
</li>

<li class="nav-item">
  <a class="nav-link" href="#">axios</a>
</li>

</ul>

<form action="">
  <div class="d-flex flex-column flex-sm-row">

    <input type="search"
      class="form-control rounded-0 me-1 mb-2 mb-sm-0"
      placeholder="input">

    <button class="btn btn-success rounded-0 mb-2 mb-sm-0">
      Submit
    </button>

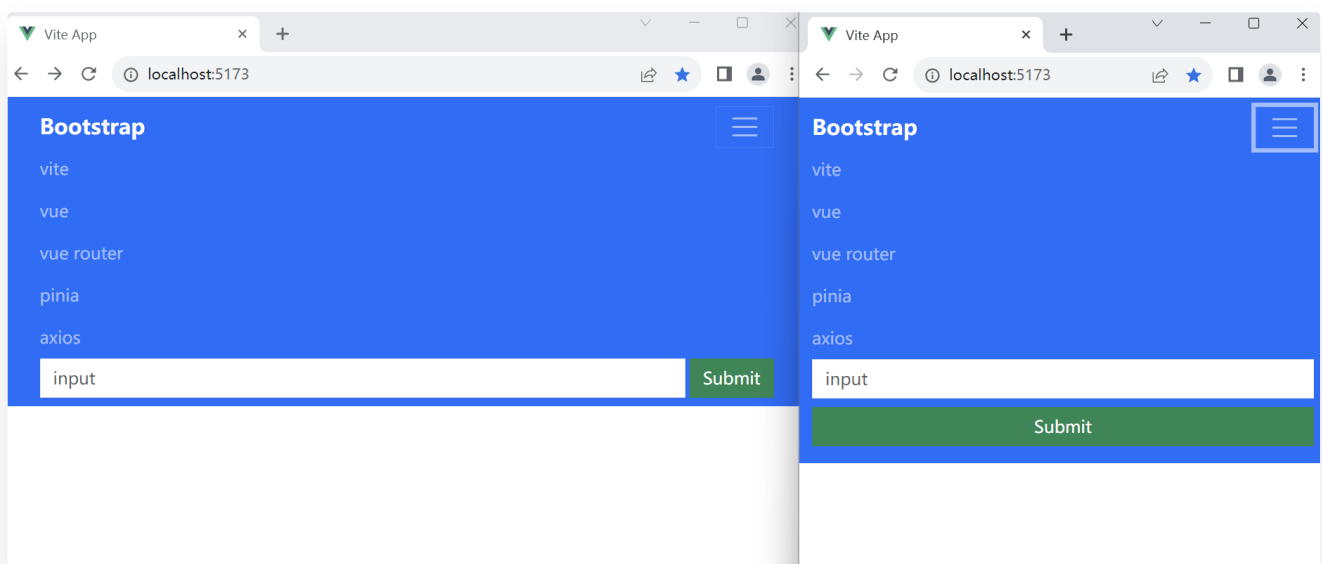
  </div>
</form>

</div>

</nav>
</header>
</template>

```

- Bootstrap UI 效果



06. TailwindCss

≡ 官网: <https://www.tailwindcss.cn/>

- TAILWIND 是一款 原子类 CSS UI 框架、预设了所有 css 属性的 class



≡ 安装参考: <https://www.tailwindcss.cn/docs/guides/vite#vue>

- npm [请先创建 vite-vue 项目]



```
npm install -D tailwindcss postcss autoprefixer
```

- npx [初始化 tailwindcss 配置文件]



```
npx tailwindcss init -p

// 该指令会自动在项目根目录创建文件 tailwind.config.js
```

- tailwind.config.js



```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{vue,js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

- ./src/style.css



```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

- main.js

```
import {createApp} from 'vue'
import App from './App.vue'

import './style.css'

const app = createApp(App)

app.mount('#app')
```

- **app.vue**

```
<template>

  <section class="bg-slate-100 rounded-xl p-8 dark:bg-slate-800">

    <div class="pt-6 space-y-4">

      <blockquote>
        <p class="text-lg font-me">
          Tailwind CSS is the only framework that I've seen scale
          on large teams. It's easy to customize, adapts to any
design,
          and the build size is tiny.
        </p>
      </blockquote>

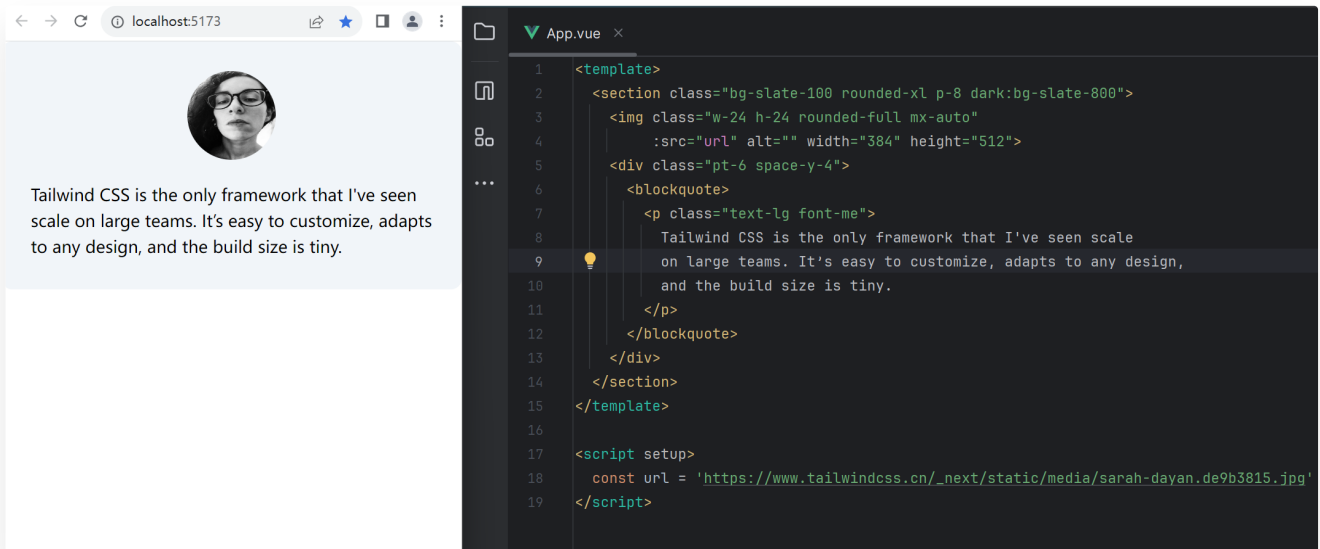
    </div>
  </section>

</template>

<script setup>
  const url = 'https://www.tailwindcss.cn/_next/static/media/sarah-
dayan.de9b3815.jpg'
```

</script>

- 效果



07. 动态样式

- 使用 class 绑定为静态绑定总是生效
- 使用 :class 绑定为动态绑定、需用对象语法 { '类名称': 布尔变量 }
- 渲染结果为组合类选择器 [类1 类N]

- app.vue

```

<template>
  <!-- 渲染结果为 [fs-60 green] -->
  <h1 class="fs-60" :class="{ 'green': isActive}">动态样式</h1>
</template>

<script setup>
  import {ref} from 'vue'
  const isActive = ref(true)
</script>
```

```
<style scoped>
  .fs-60 {
    font-size: 60px;
  }

  .green {
    color: #65b687;
  }
</style>
```

03. 组件高级

≡ 组件高级涵盖：组件注册引用、组件插槽、动态组件、组件缓存、生命周期等

3.1 组件引用与注册

≡ 不同业务功能应当由不同组件完成、开发者应当有意识分解功能到不同组件

- ① vue 组件 应当存放于 ./src/components/目录下
- ② vue 组件 文件名称可用首字母大写的驼峰规则命名、或 短横线-间隔单词命名
- ③ vue 组件 引用时无论文件名称命名方式、都可 以驼峰 或 短横线单词引用

01. 组件引用

≡ 组件可像 HTML 元素那样被嵌入其它组件的 `template` 结构中

- 组件引用时可使用驼峰、或短横线、且必须闭合或自闭合方式引用
- 组件可任意多次引用
- 当前使用其他组件的组件称为父组件、被引用的组件称为子组件

- `src/components/FirstVueChild.vue`

```
● ● ●  
  
<template>  
  <h1>vue-first-child</h1>  
</template>
```

- `app.vue`

```
● ● ●  
  
<template>  
  <h1>Vue 组件引用</h1>  
  <FirstVueChild></FirstVueChild>  
  <FirstVueChild/>  
  <first-vue-child></first-vue-child>  
  <first-vue-child/>  
</template>  
  
<script setup>  
  import FirstVueChild from '@components/FirstVueChild.vue'  
</script>
```

02. 组件注册

≡ 根组件外、其它组件使用时必须注册、组件注册分为、局部被动注册和全局注册

- 采用 `import 组件名 from '@components/组件名.vue'` 为局部被动注册
- 局部注册方式的组件仅在当前组件内生效、其它组件使用时仍需导入

- **app.vue**

```
● ● ●  
  
<template>  
  <h1>Vue 组件注册</h1>  
  <vue-first-child/>  
</template>  
  
<script setup>  
  import FirstVueChild from '@components/FirstVueChild.vue'  
</script>
```

- 全局注册需在 `main.js` 中完成、全局注册后的组件可在任意组件中直接使用

- **main.js**



```
import {createapp} from 'vue'
import app from './app.vue'

import FirstVueChild from '@components/FirstVueChild.vue'

const app = createapp(app)

// 此处进行全局注册 app.component(组件名称、组件对象)
app.component('FirstVueChild', FirstVueChild)
  .component(...) 此处表示以此类推

app.mount('#app')
```

- **app.vue**



```
<template>
  <h1>组件全局注册后可直接使用</h1>
  <first-vue-child/>
</template>

<script setup>
  // 无需再进行导入 FirstVueChild 组件
</script>
```

3.2 组件传值

≡ Vue 允许父子组件间单向传递数据、以及跨级传递数据

01. 父传子

≡ 父传子数据流向为单向、子组件需定义接收数据、父组件绑定自定义属性

- 传值逻辑：
- 子组件 `defineProps({属性名: {数据类型及可选属性}})` 表示子组件接收属性
- 父组件 `<子组件名称 :属性名称="变量值">`
- `defineProps` 是 `vue` 全局宏函数可在 `<script setup>` 中直接使用

- `app.vue`

```

● ● ●
<template>
  <h1>父组件</h1>
  <h1>numberRef : {{ numberRef }}</h1>
  <h1>user : {{ user }}</h1>
  <child message="this i a text."
    :numberRef="numberRef"
    :user="user" />
</template>

<script setup>
  import Child from '@components/Child.vue'
  import {ref, reactive} from 'vue'

  const numberRef = ref(1000)

  const user = reactive({
    username: 'lucas',
    gender: 'male'
  })
</script>

<style scoped>
  h1 {
    padding-left: 20px;
    color: #65b687;
  }

```

```
</style>
```

- `./src/components/Child.vue`

```
● ● ●

<template>
  <section>
    <h1>子组件</h1>
    <h2>message : {{ message }}</h2>
    <h2>numberRef : {{ numberRef }}</h2>
    <h2>user : {{ user }}</h2>
    <h3>{{ props }}</h3>
  </section>
</template>

<script setup>
  const props = defineProps({
    message: {type: String, required: true},
    numberRef: {type: Number, required: true},
    user: {type: Object, required: true}
  })
</script>

<style scoped>
  section {
    width: fit-content;
    padding: 20px;
    background-color: #eee;
  }
</style>
```

- 效果

父组件

numberRef : 1000

user : { "username": "lucas", "gender": "male" }

子组件

message : this i a text.

numberRef : 1000

user : { "username": "lucas", "gender": "male" }

{ "message": "this i a text.", "numberRef": 1000, "user": { "username": "lucas", "gender": "male" } }

≡ 特殊的：引用子组件可使用 `class` 绑定样式、将被绑定到子组件根元素上

- `app.vue`

```
● ● ●  
  
<template>  
  <h1>父组件</h1>  
  <Child message="this i a text."  
    :numberRef="numberRef"  
    :user="user"  
    class="title"/>  
  
</template>
```

02. 子传父

子组件定义自定义事件、父组件绑定该事件通过处理函数的参数获取数据

- 子组件定义自定义事件使用宏函数：`defineEmits(['事件名称'])`
- `./src/components/Child.vue`

```

<template>
  <section>
    <h1>子组件</h1>
    <h2>{{ numberRef }}</h2>
    <h2>{{ user }}</h2>
    <h1>
      <button @click="transfer">点我发送数据</button>
    </h1>
  </section>
</template>

<script setup>
  import {ref} from 'vue'

  // 定义自定义事件 onOk
  const emits = defineEmits(['onOk'])

  const numberRef = ref(100)

  const user = ref({
    username: 'lucas',
    gender: 'male'
  })

  function transfer() {
    // 向 onOk 事件提交数据
    emits('onOk', numberRef, user)
  }
</script>

<style scoped>
  section {
    width: fit-content;
    padding: 20px;
  }
</style>
```

```
        background-color: #eee;
    }
</style>
```

- **app.vue**

```


<template>
  <h1>父组件</h1>
  <h1>number : {{ number }}</h1>
  <h1>user : {{ user }}</h1>

  <!-- 绑定onOk自定义事件 -->
  <child @onOk="receive"/>
</template>

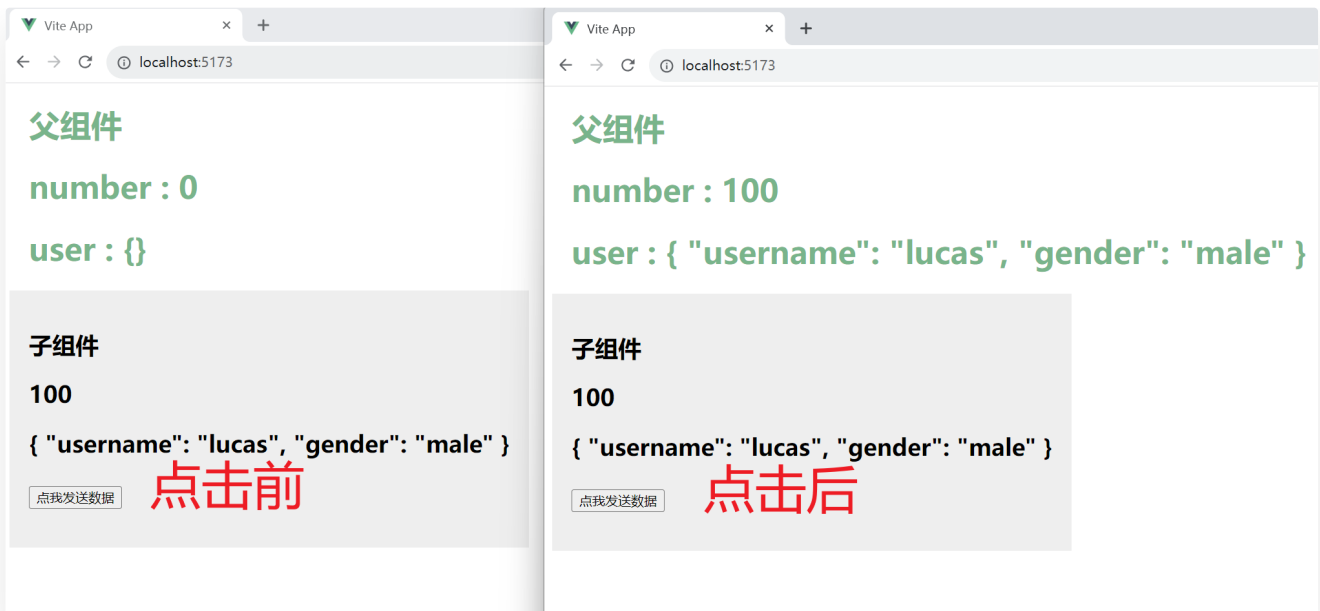
<script setup>
  import {ref} from 'vue'
  import child from '@components/Child.vue'

  const number = ref(0)
  const user = ref({})

  function receive(arg1, arg2) {
    number.value = arg1
    user.value = arg2
  }
</script>

<style scoped>
  h1 {
    padding-left: 20px;
    color: #65b687;
  }
</style>
```

- **效果**



脚心留心

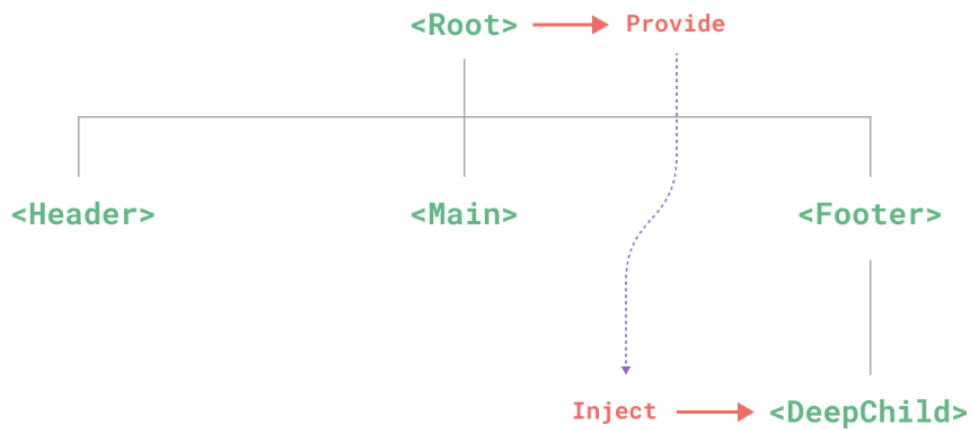
- 子传父需要调用 `emits('事件', 数据..)` 方式进行数据传递
- 若想主动触发数据传递、可使用响应式数据、对该数据进行 `watch` 监听即可

03. 依赖注入

≡ 依赖注入允许数据跨级传递、但仅允许高 → 低传递 [不支持反向流转]

- `import { provide, inject } from 'vue'`
- 父级组件: `provide('key', data)`
- 儿孙组件: `const data = inject('key')`

- 数据流转关系



- **app.vue**

```

<template>
  <section>
    <h1>根组件 : {{ number }}</h1>
    <parent></parent>
  </section>
</template>
<script setup>
  import Parent from '@components/Parent.vue'
  import {ref, provide} from 'vue'

  const number = ref(Math.random())

  setInterval(() => number.value = Math.random(), 2000)

  provide('number', number)
</script>

```

- **./src/components/Parent.vue**



```
<template>
  <section>
    <h1>父组件</h1>
    <child></child>
  </section>
</template>

<script setup>
  import Child from '@components/Child.vue'
</script>
```

- ./src/components/Child.vue



```
<template>
  <h1>子组件 : {{ number }}</h1>
</template>

<script setup>
  import {inject, isRef} from 'vue'
  const number = inject('number')
  console.log(isRef(number))
</script>
```

- 效果

根组件 : 0.9279196392322444

父组件

子组件 : 0.9279196392322444

04. 状态管理

- 当多个组件没有关系、或关系结构复杂时又依赖相同数据、则可使用状态管理
- 状态管理的数据应当作为全局数据使用、如用户登录状态及用户信息
- 请参考后续《状态管理》章节

3.3 动态组件

≡ 组件也可使用 `v-if` 控制显式或隐藏、但 `vue` 提供针对机制以获得更多控制行为

- `<component :is="组件名称" />` [`component` 是 `vue` 内置类似组件的元素]
- 未在 `:is="组件名称"` 中显式的组件、将被卸载

- `./src/components/ButtonLeft.vue`

```
● ● ●  
  
<template>  
  <button>左按钮</button>  
</template>  
  
<style scoped>  
  button {  
    all: unset;  
    padding: 10px 30px;  
    font-size: 18px;  
    color: #fff;  
    background-color: #65b687;  
  }  
</style>
```

- `/src/components/ButtonRight.vue`



```
<template>
  <button>右按钮</button>
</template>

<style scoped>
  button {
    all: unset;
    padding: 10px 30px;
    font-size: 18px;
    color: #fff;
    background-color: #da0052;
  }
</style>
```

- **app.vue**



```
<template>
  <div>
    <h1>
      动态组件<button @click="showLeft = !showLeft">点击切换</button>
    </h1>

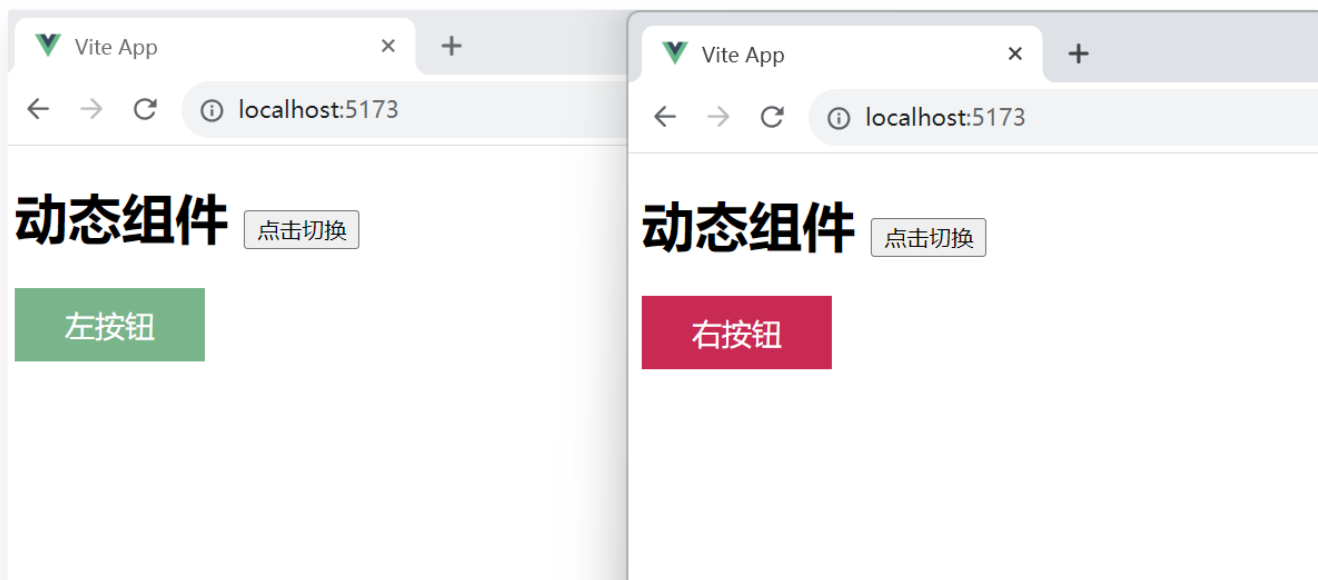
    <component :is="showLeft ? ButtonLeft : ButtonRight" />
  </div>
</template>

<script setup>
  import ButtonLeft from '@components/ButtonLeft.vue'
  import ButtonRight from '@components/ButtonRight.vue'

  import {ref} from 'vue'

  const showLeft = ref(true)
</script>
```

- 效果



3.4 内置组件

vue 提供了几个同开发者开发一样的组件、内置 vue 全局中可直接使用

01. 组件缓存

对于动态组件切换时将被卸载、因为丢失组件操作状态

- `<KeepAlive>` 动态组件 `</KeepAlive>`
- `<KeepAlive>` 组件包裹的组件将总是被缓存、切换回时能保持组件操作状态
- `<KeepAlive>`
 - `include="指定缓存的组件名1 名N"`
 - `exclude="排除缓存的组件名1 名N"`
 - `max="最大缓存组件个数N"`
- `<KeepAlive>` 甚至还会缓存 鼠标、滑动条位置、即当前组件所有状态

- ./src/components/ButtonLeft.vue

```
• • •

<template>
  <p>counter : {{counter}}</p>
  <button @click="counter++">左按钮</button>
</template>

<script setup>
  import {ref} from 'vue'
  const counter = ref(0)
</script>

<style scoped>
  button {
    all: unset;
    padding: 10px 30px;
    font-size: 18px;
    color: #fff;
    background-color: #65b687;
  }
</style>
```

- 效果

动态组件

点击切换

counter : 10

左按钮

对counter 累加后、点击切换回时 counter = 0

- app.vue [keep-alive]



```
<template>
  <div>
    <h1>动态组件
    <button @click="showLeft = !showLeft">点击切换</button>
    </h1>
    <keep-alive>
      <component :is="showLeft ? ButtonLeft : ButtonRight"/>
    </keep-alive>
  </div>
</template>

<script setup>
  import ButtonLeft from '@components/ButtonLeft.vue'
  import ButtonRight from '@components/ButtonRight.vue'
  import {ref} from 'vue'

  const showLeft = ref(true)
</script>
```

02. 过渡动画

≡ Transition 内置组件提供轻微的动画过渡效果、且仅在以下行为触发

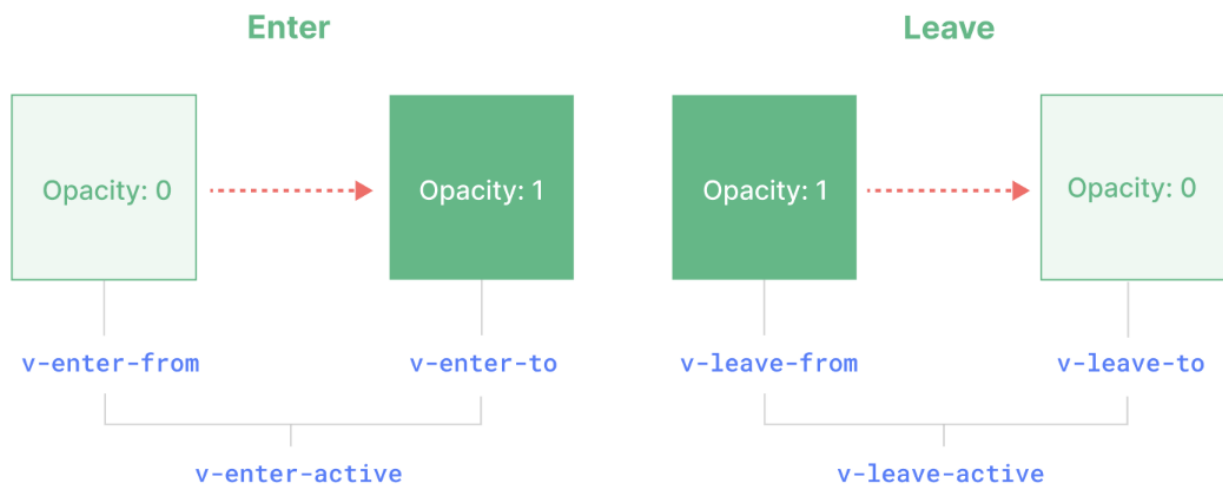
- 由 `v-if` 所触发的切换
- 由 `v-show` 所触发的切换
- 由特殊元素 `<component>` 切换的动态组件
- 改变特殊的 `key` 属性
- `.vue`



```
<transition>
  <h1 v-if="show">内容 </h1>
</transition>
```

- 预置 css 类选择器

一共有 6 个应用于进入与离开过渡效果的 CSS class。



- app.vue [slide-fade 为自定义名称]



```
<template>
  <div class="container py-5">
    <button @click="isActive = !isActive"
      class="btn btn-success btn-lg w-25 rounded-0 mb-3">过渡动画
    </button>
    <transition name="slide-fade">
      <h1 v-if="isActive" class="py-5 text-bg-secondary text-center">
        <div v-pre class="py-5">
          <div class="py-5">
            {{ 动画使用 }}
          </div>
        </div>
      </h1>
```

```
    </transition>
  </div>
</template>

<script setup>
  import {ref} from 'vue'
  const isActive = ref(true)
</script>

<style scoped>
  .slide-fade-enter-active {
    transition: all 0.3s ease-out;
  }

  .slide-fade-leave-active {
    transition: all 0.8s cubic-bezier(1, 0.5, 0.8, 1);
  }

  .slide-fade-enter-from,
  .slide-fade-leave-to {
    transform: translateX(20px);
    opacity: 0;
  }
</style>
```

3.5 组件插槽

≡ 插槽是当前组件预留 HTML 空位的行为、在引用时插入具体内容

01. 单一插槽

≡ 单一插槽是指当前组件仅预留了一个空位、当填入内容总是填入到该位置

- `./src/components/Header.vue`



```
<template>
  <div class="container py-5">
    <h1 class="text-center text-secondary display-3 fw-bold">
      千里之行，始于足下。
    </h1>
    <!-- 此处是插槽位置 -->
    <slot></slot>
  </div>
</template>

<script setup>
  import 'bootstrap/scss/bootstrap.scss'
</script>
```

- `app.vue`



```
<template>
  <!-- 此处不使用插槽 -->
  <Header />

  <Header>
    <!-- 此处是使用 Header 组件时 插入插槽 -->
    <p class="text-center text-danger">
      A thousand mile trip begins with one step.
    </p>

    <p class="text-center text-danger">{{ author }}</p>
  </Header>
</template>

<script setup>
  import Header from '@components/Header.vue'
  import {ref} from 'vue'
  const author = ref('天气预报')
</script>
```

02. 具名插槽

具名插槽是指当前组件仅预留了N个空位、当填入内容时需指定位置填入

- 必须使用 `template #插槽名称` 指定插入那个插槽位置
- `./src/components/Header.vue`

```
  
<template>  
  <div class="container py-5">  
  
    <h1 class="text-center text-secondary fw-bold">项目1</h1>  
  
    <slot name="s1"></slot>  
  
    <h1 class="text-center text-secondary fw-bold">项目2</h1>  
  
    <slot name="s2"></slot>  
  
  </div>  
</template>  
  
<script setup>  
  import 'bootstrap/scss/bootstrap.scss'  
</script>
```

- `app.vue`

```
  
<template>  
  <Header>  
  
    <template #s1>  
      <p class="text-center">项目内容1</p>
```

```

    <p class="text-center">项目内容2</p>
    <p class="text-center">项目内容3</p>
    <p class="text-center text-danger">{{ author1 }}</p>
  </template>

  <template #s2>
    <p class="text-center">项目内容21</p>
    <p class="text-center">项目内容22</p>
    <p class="text-center">项目内容23</p>
    <p class="text-center text-danger">{{ author2 }}</p>
  </template>

</Header>
</template>

<script setup>
  import Header from '@components/Header.vue'

  import {ref} from 'vue'

  const author1 = ref('天气预报')
  const author2 = ref('北极圈有环流')
</script>

```

03. 循环插槽

≡ v-for 结构中允许循环插槽结构

- 显然插槽内容是父组件填充的、此时填充的数据也应该随着插槽填充
- 插槽需绑定当前循环的变量 <slot :index="index" />
- 父组件填充插槽必须使用 <template #default="{index}"> 绑定循环变量 #绑定id
- ./src/components/Child.vue



```
<script setup>
defineProps({
  title: {type: String, required: true, default: '标题占位'},
  counter: {type: Number, required: true, default: 4}
})

</script>
<template>

  <div class="container py-3">
    <h1 class="text-center mb-5 py-3
      border-bottom border-2 border-dark-subtle text-
secondary fw-bold">

      {{ title }}
    </h1>
    <div class="row row-cols-xl-4 row-cols-lg-3 row-cols-md-2 row-
cols-1">

      <div class="col mb-5" v-for="index in counter" :key="index">
        <div class="card mx-auto rounded-0">
          <div class="bg-secondary-subtle" style="height: 200px;">
</div>
          <div class="card-body">
            <h5 class="placeholder-glow">
              <span class="placeholder col-6"></span>
            </h5>
            <slot :index="index" />
            <a href="" class="btn btn-primary disabled placeholder
col-6"></a>
          </div>
        </div>
      </div>
    </div>
  </div>

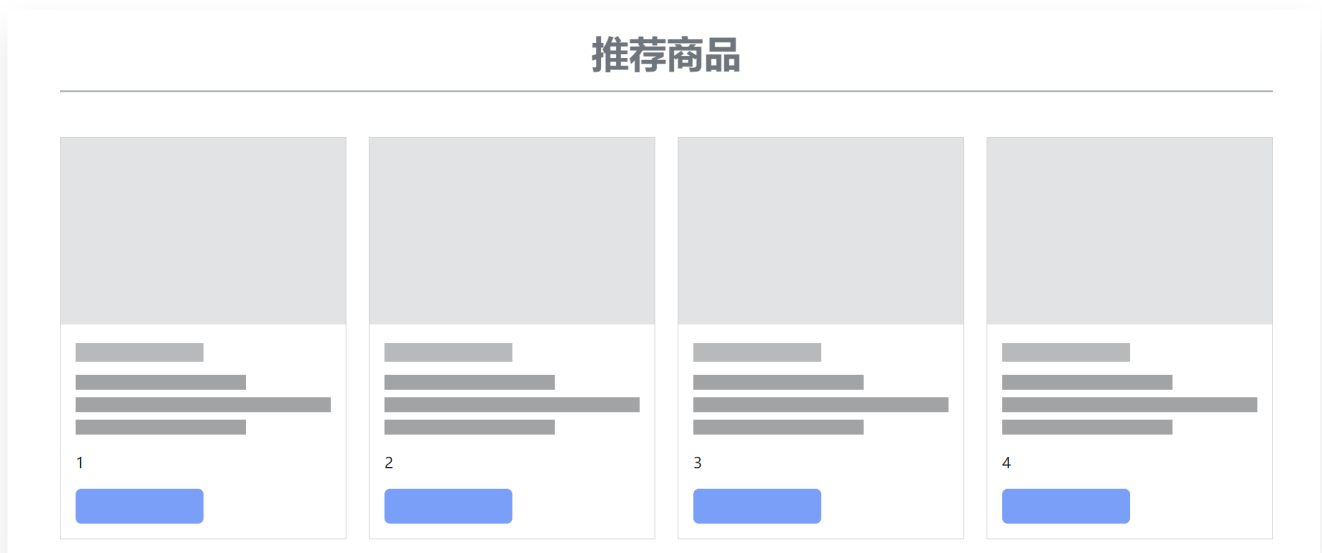
</template>

<style>
.card {
  max-width: 380px;
}
</style>
```

- app.vue

```
• • •  
  
<template>  
  <Child :counter="16" :title="'推荐商品'">  
    <template #default="{index}">  
      <div class="placeholder-glow mb-3">  
        <span class="placeholder col-8"/>  
        <span class="placeholder col-5"/>  
        <span class="placeholder col-7"/>  
        <span class="placeholder col-8"/>  
      </div>  
      <p>{{ index }}</p>  
    </template>  
  </Child>  
</template>  
  
<script setup>  
  import Child from '@components/Child.vue'  
</script>
```

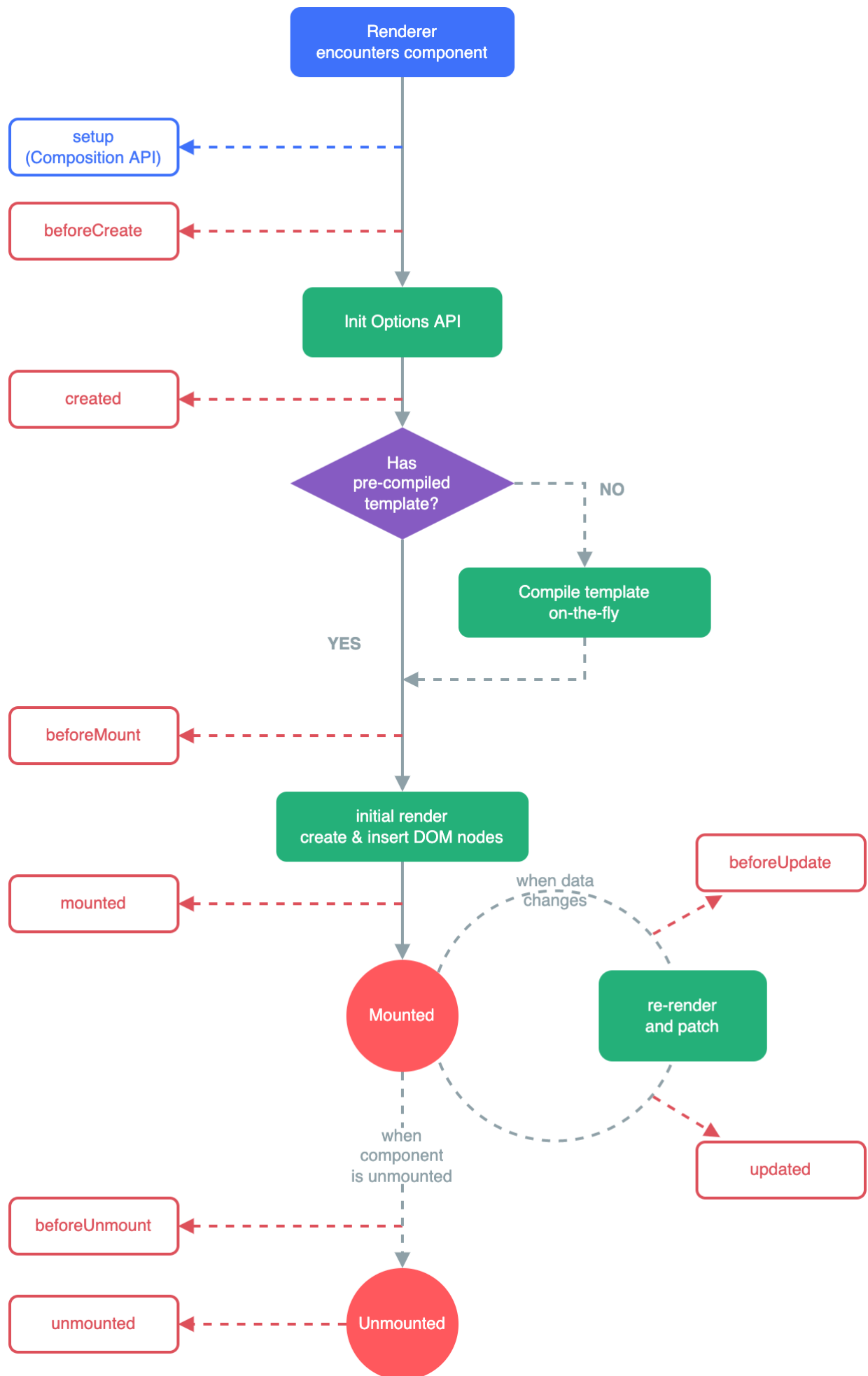
- 效果



3.6 组件生命周期

≡ vue 在将组件转为 DOM 并渲染到 HTML 中时、预留了钩子函数

- 这些刻意预留的钩子函数被称为生命周期钩子、会在 **对应时机主动** 执行
- 生命周期流程参考



01. onMounted

≡ 注册一个回调函数、在组件挂载完成后执行



```
function onMounted(callback: () => void): void
```

- app.vue



```
<template>
  <h1 v-pre>生命周期钩子: {{ onMounted }} </h1>
</template>

<script setup>
  import {onMounted} from 'vue'

  onMounted(() => console.log('onMounted...'))
</script>
```

02. onUpdated

≡ 注册一个回调函数、在组件因为响应式状态变更而更新其 DOM 树之后调用



```
function onUpdated(callback: () => void): void
```

- `app.vue`

```
● ● ●  
  
<template>  
  <h1 v-pre>生命周期钩子: {{ onUpdated }} </h1>  
  <h1>number : {{number}}</h1>  
</template>  
  
<script setup>  
  import {onUpdated, ref} from 'vue'  
  
  onUpdated(() => console.log('onUpdated...'))  
  const number = ref(Math.random())  
  
  // 此处 每次 number 值的变更都会触发 onUpdated 的执行  
  setInterval(() => number.value = Math.random(), 1000)  
</script>
```

03. 其它

- 请参考: <https://cn.vuejs.org/api/composition-api-lifecycle.html>

04. Axios

4.1 安装

≡ axios 异步网络请求 <https://www.axios-http.cn/docs/intro>

- axios 中文网



起步

基于promise可以用于浏览器和node.js的网络请求库

Axios 是什么？

Axios 是一个基于 [promise](#) 网络请求库，作用于 [node.js](#) 和浏览器中。它是 [isomorphic](#) 的(即同一套代码可以运行在浏览器和node.js中)。在服务端它使用原生 node.js [http](#) 模块, 而在客户端 (浏览器端) 则使用 XMLHttpRequests。

特性

- 从浏览器创建 [XMLHttpRequests](#)
- 从 node.js 创建 [http](#) 请求
- 支持 [Promise](#) API
- 拦截请求和响应
- 转换请求和响应数据
- 取消请求
- 超时处理
- 查询参数序列化支持嵌套项处理
- 自动将请求体序列化为:
 - JSON ([application/json](#))
 - Multipart / FormData ([multipart/form-data](#))
 - URL encoded form ([application/x-www-form-urlencoded](#))
- 将 HTML Form 转换成 JSON 进行请求
- 自动转换JSON数据
- 获取浏览器和 node.js 的请求进度，并提供额外的信息（速度、剩余时间）

• Npm



```
npm install axios
```

```
npm install qs
```

4.2 GET/POST

• server.js [node ./server.js]



```
const http = require('node:http')
```

```
const server = http.createServer((request, response) => {
```

```

    console.log(request.url)

    response.setHeader('content-type', 'application/json')
    response.setHeader('Access-Control-Allow-Origin', "*")

    const method = request.method

    if (method === 'GET') {
      const user = {
        username: 'lucas',
        gender: 'male',
        method: 'GET'
      }
      response.end(JSON.stringify(user))
    }

    if (method === 'POST') {
      const user = {
        username: 'jerry',
        gender: 'female',
        method: 'POST'
      }
      response.end(JSON.stringify(user))
    }
  })

  const port = 3000

  server.listen(port, () => console.log(`server running on port :
  ${port}`))

```

- app.vue [GET]

```

<template>
  <h1>userRef: {{ userRef }}</h1>
  <h1>axios GET</h1>
  <h1>
    <button @click="get">GET</button>
  </h1>
</template>

<script setup>

```

```

import {ref} from 'vue'
import axios from 'axios'
import qs from 'qs'

const userRef = ref({})

function get() {

  const user = {username: 'lucas', gender: 'male'}

  axios.get('http://localhost:3000/get?' + qs.stringify(user))
    .then(response => response.data)
    .then(data => userRef.value = data)
}
</script>

```

- app.vue [POST]

```

● ● ●

<template>
  <h1>userRef: {{ userRef }}</h1>
  <h1>axios POST</h1>
  <h1>
    <button @click="post">POST</button>
  </h1>
</template>

<script setup>
  import {ref} from 'vue'
  import axios from 'axios'
  import qs from 'qs'

  const userRef = ref({})

  function post() {
    const user = {username: 'lucas', gender: 'male'}

    // POST 请求时 参数为 url, 请求参数
    axios.post('http://localhost:3000/post', qs.stringify(user))
      .then(response => response.data)
      .then(data => userRef.value = data)
  }
</script>

```

4.3 封装

- ./src/api/index.js

```
import axios from 'axios'

const baseURL = 'http://localhost:3000'

const request = axios.create({
  baseURL: baseURL
})

// 请求拦截
request.interceptors.request.use(function (config){

  console.log(config)
  console.log('before request do something...')

  return config

}, function (error){ return Promise.reject(error) })

// 响应拦截
request.interceptors.response.use(response => {

  console.log('before response data do something...')

  return response.data

}, error => Promise.reject(error))

export default request
```

- app.vue

```
<template>
  <h1>userRef: {{ userRef }}</h1>
```

```
<h1>axios GET</h1>
<h1>
  <button @click="get">GET</button>
</h1>
</template>

<script setup>
  import {ref} from 'vue'

  import request from '@api/index.js'
  import qs from 'qs'

  const userRef = ref({})

  function get() {
    request.get('/get' + '?' + qs.stringify({username: 'lucas'}))
      .then(data => userRef.value = data)
  }
</script>
```

05. Vue Router

≡ 官网: <https://router.vuejs.org/>

- vue router 路由是 vue 官方独立技术、用于控制视图跳转



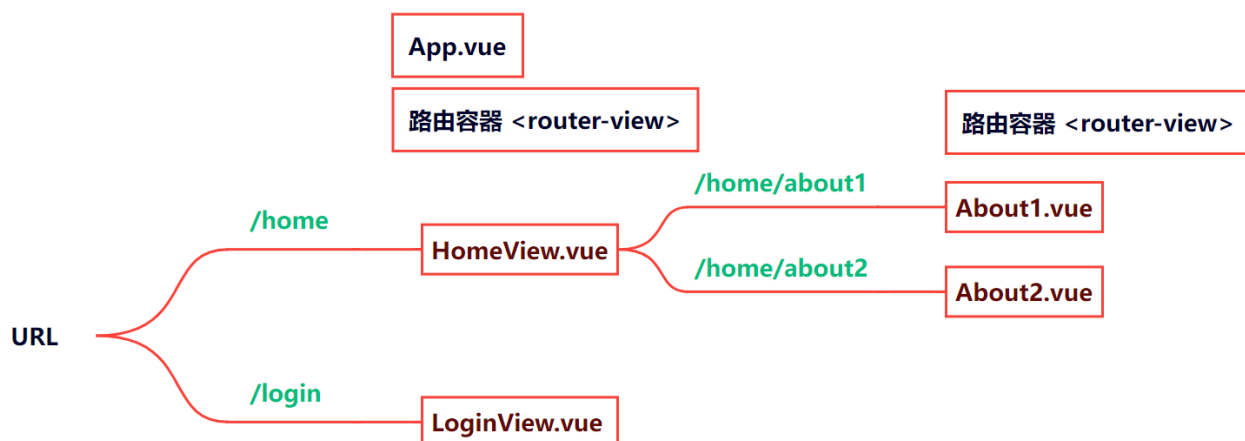
5.1 路由概念机制

路由用于控制 URL 和 组件的显式关系

- 路由在逻辑上：分为通过 URL 访问某个页面、或访问某个页面中的组件
- 路由在使用上：分为声明式路由、及程式化路由

- 路由逻辑导航示意图

Vue Router 路由导航示意图



说明

- ① 使用路由需要指定 URL 和 组件的对应关系、如访问 `/home` → 渲染 `HomeView`
- ② 渲染路由 URL 对应的组件时还需要指定 路由容器 [组件究竟在哪儿显式]
- ③ 显然、想要跳转整个页面、必须是一级路由、跳转组件则需是子路由
- ④ vue 总是从根组件开始渲染、因此跳转页面的路由容器必须是根组件

5.2 安装

```
npm install vue-router
```

- npm



```
npm install vue-router
```

- 路由配置文件通常约定于 `./src/router/index.js` [该目录及文件名称可自定义]
- `./src/router/index.js`

```
import {createRouter, createWebHistory} from 'vue-router'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/home',
      name: 'home',
      component: () => import('@/views/HomeView.vue')
    },
    {
      path: '/login',
      name: 'login',
      component: () => import('@/views/LoginView.vue')
    }
  ]
})

export default router
```

说明

- ① `createRouter({})` 函数用于创建路由配置
- ② `history`: 属性表示路由浏览器历史前后退规则
- ③ `routes`: 属性是数组、里面 **每一个路由对象对应一个路由规则**
 - `path`: 表示 URL
 - `name`: 表示 URL 逻辑名称
 - `component`: 表示路由对应的组件
 - `meta`: { 路由元信息、需自定义且自行在路由 api 中读取 }

- main.js [路由规则并不需要调用执行、挂载 main.js 上即可]

```
import {createApp} from 'vue'
import App from './App.vue'

import router from '@router/index.js'

import 'bootstrap/scss/bootstrap.scss'

const app = createApp(App)

app.use(router).mount('#app')
```

5.3 路由导航

01. 路由页面

≡ 路由页面需将 App.vue 作为路由容器

- <router-view/> 指定路由组件显式位置
- app.vue

```
<template>
  <router-view/>
</template>
```

- `app.vue` [带缓存的路由控制]



```
<!-- 推荐下带有缓存的路由控制 -->
<template>
  <router-view v-slot="{ Component }">
    <keep-alive>
      <component :is="Component"/>
    </keep-alive>
  </router-view>
</template>
```

- `./src/views/HomeView.vue`



```
<template>
  <div class="container mt-5 py-5 bg-primary">
    <h1 class="text-center text-white fw-bold">Home</h1>
  </div>
</template>
```

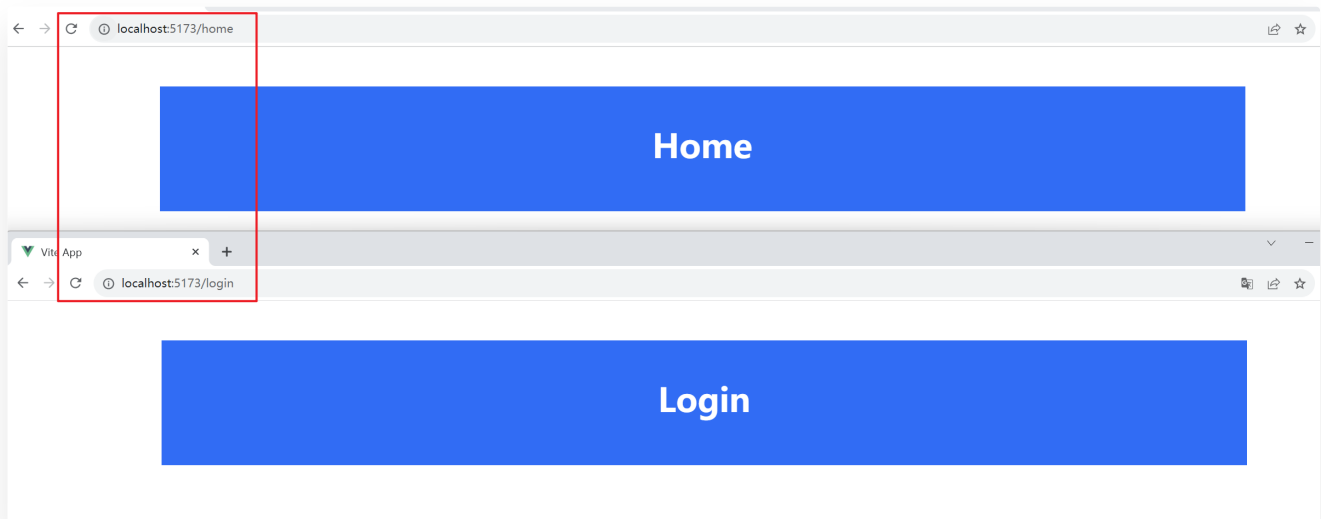
- `./src/views/LoginView.vue`



```
<template>
  <div class="container mt-5 py-5 bg-primary">
    <h1 class="text-center text-white fw-bold">Login</h1>
  </div>
</template>
```

- <http://localhost:5173/home>

- <http://localhost:5173/login>
- 效果



02. 路由重定向

≡ URL 重定向是 HTTP 协议的规则、重定向时为二次请求

- `./src/router/index.js`

```
import {createRouter, createWebHistory} from 'vue-router'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      redirect: '/home'
    },
    {
      path: '/home',
      name: 'home',
      component: () => import('@views/HomeView.vue')
    },
    {
      path: '/login',
      name: 'login',
    }
  ]
})
```

```

        component: () => import('@/views/LoginView.vue')
      }
    ]
  })

export default router

```

03. 404路由

≡ 404 路由特用 `path: '/*:pathMatch(.*)'` 配置、表示匹配不上的路由

- `./src/router/index.js`

```

import {createRouter, createWebHistory} from 'vue-router'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      redirect: '/home'
    },
    {
      path: '/*:pathMatch(.*)',
      redirect: '/home'
    },
    {
      path: '/home',
      name: 'home',
      component: () => import('@/views/HomeView.vue')
    },
    {
      path: '/login',
      name: 'login',
      component: () => import('@/views/LoginView.vue')
    }
  ]
})

```

```
export default router
```

5.4 路由组件及子路由

≡ 路由访问组件时、需先访问到对应页面、当前页面路由为父路由

01. 路由组件页面

- `./src/views/HomeView.vue`

```
● ● ●  
<template>  
  <div class="container mt-5 py-5 bg-primary">  
    <h1 class="text-center text-white fw-bold">Home</h1>  
  
    <div class="d-flex justify-content-md-center gap-3">  
      <button class="btn btn-danger">关于1</button>  
      <button class="btn btn-success">关于2</button>  
    </div>  
  
    <div class="p-5 mt-5 bg-secondary-subtle"></div>  
  
  </div>  
</template>
```

- 效果 [`localhost:5173/home`]

Home

[关于1](#)[关于2](#)

此处是预留路由容器

02. 路由组件

- `./src/components/About1.vue`



```
<template>
  <h1 class="py-5 text-center">About1 组件</h1>
</template>
```

- `./src/components/About1.vue`



```
<template>
  <h1 class="py-5 text-center">About2 组件</h1>
</template>
```

03. router-link

≡ `<router-link>` 用于控制点击跳转的组件、 `router-link` 被渲染为a元素

- router-link 被渲染为 a 元素、和 UI 框架集成时、直接作用 class 即可

- ./src/views/HomeView.vue



```
<template>
  <div class="container mt-5 py-5 bg-primary">
    <h1 class="text-center text-white fw-bold">Home</h1>

    <div class="d-flex justify-content-md-center gap-3">

      <router-link to="/home/about1" class="btn btn-danger">
        关于1
      </router-link>

      <router-link to="/home/about2" class="btn btn-success">
        关于2
      </router-link>

    </div>

    <div class="p-5 mt-5 bg-secondary-subtle">

      <!-- 此处为当前路由组件的容器 -->
      <router-view />
    </div>

  </div>
</template>
```

04. 子路由配置

- ./src/router/index.js



```
import {createRouter, createWebHistory} from 'vue-router'
```

```

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      redirect: '/home'
    },
    {
      path: '/*',
      redirect: '/home'
    },
    {
      path: '/home',
      name: 'home',
      component: () => import('@/views/HomeView.vue'),
      children: [
        {
          path: '/home/about1',
          name: 'about1',
          component: () => import('@/components/About1.vue')
        },
        {
          path: '/home/about2',
          name: 'about2',
          component: () => import('@/components/About2.vue')
        }
      ]
    },
    {
      path: '/login',
      name: 'login',
      component: () => import('@/views/LoginView.vue')
    }
  ]
})

export default router

```

- 效果 [localhost:5731/home/about2]

Home

关于1

关于2

点击切换

About2 组件

05. 多router-view

≡ 页面中多个 router-view 时、需为其指定 name 属性、以致当前路由知道路由显式位置

- ./src/views/HomeView.vue

```

<template>
  <div class="text-bg-primary py-5 container">
    <h1 class="text-center">多个 router-view </h1>

    <ul class="list-unstyled">
      <li>
        <router-link to="/home/r1" class="text-white">路由1 到第一个
      </router-link>
      </li>
      <li>
        <router-link to="/home/r2" class="text-white">路由2 到第一个
      </router-link>
      </li>
      <li>
        <router-link to="/home/r3" class="text-white">路由3 到第二个
      </router-link>
      </li>
    </ul>
  </div>
</template>

```

```

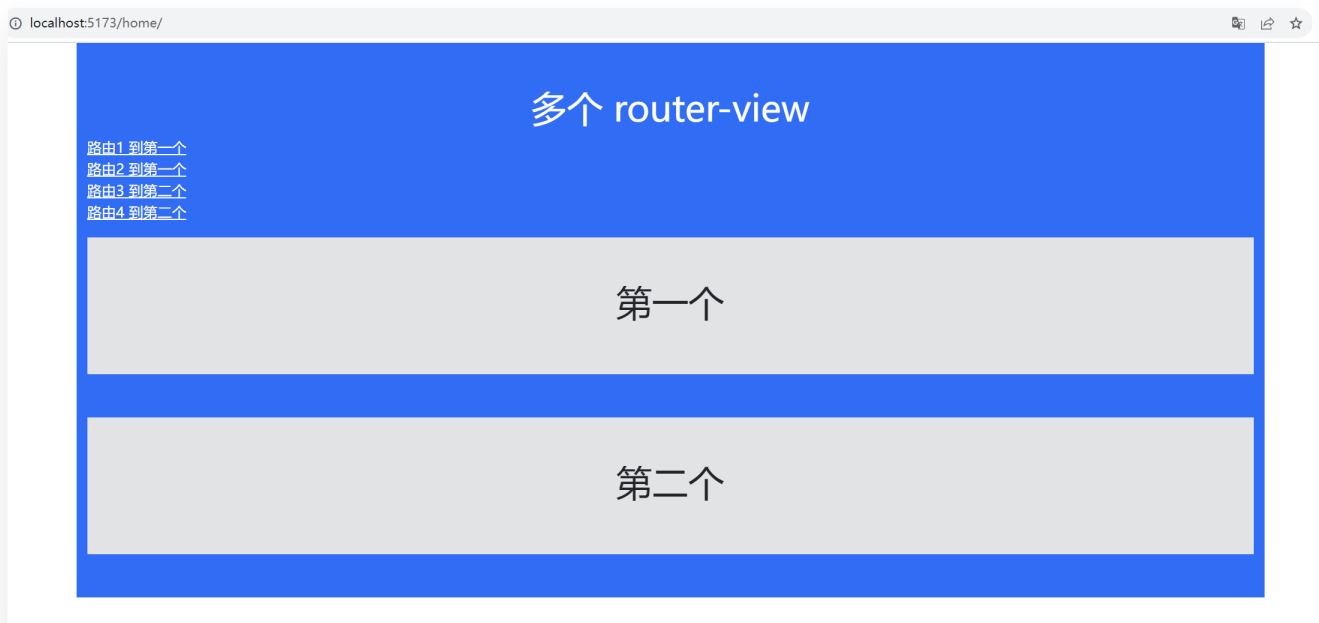
        <router-link to="/home/r4" class="text-white">路由4 到第二个
    </router-link>
  </li>
</ul>

  <div class="bg-secondary-subtle py-5">
    <h1 class="text-center text-dark">第一个</h1>
    <router-view name="one" />
  </div>
  <div class="bg-secondary-subtle py-5 mt-5">
    <h1 class="text-center text-dark">第二个</h1>
    <router-view name="two" />
  </div>

</div>
</template>

```

- 效果



- ./src/components/R1 R2 R3 R4.vue

```

  ● ● ●

  <template>
    <h1 class="py-5 text-bg-danger text-center">R1</h1>
  </template>

```

```

<template>
  <h1 class="py-5 text-bg-danger text-center">R2</h1>
</template>

<template>
  <h1 class="py-5 text-bg-danger text-center">R3</h1>
</template>

<template>
  <h1 class="py-5 text-bg-danger text-center">R4</h1>
</template>

```

- `./src/router/index.js`



```

import {createRouter, createWebHistory} from 'vue-router'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      redirect: '/home'
    },
    {
      path: '/home',
      name: 'home',
      component: () => import('@views/HomeView.vue'),
      children: [
        {
          path: '/home/r1',
          name: 'r1',

          // 常规配置
          // component: () => import('@components/R1.vue'),

          // 指定 router-name 名称配置

          components: {
            one: () => import('@components/R1.vue')
          }
        },
        {
          path: '/home/r2',

```

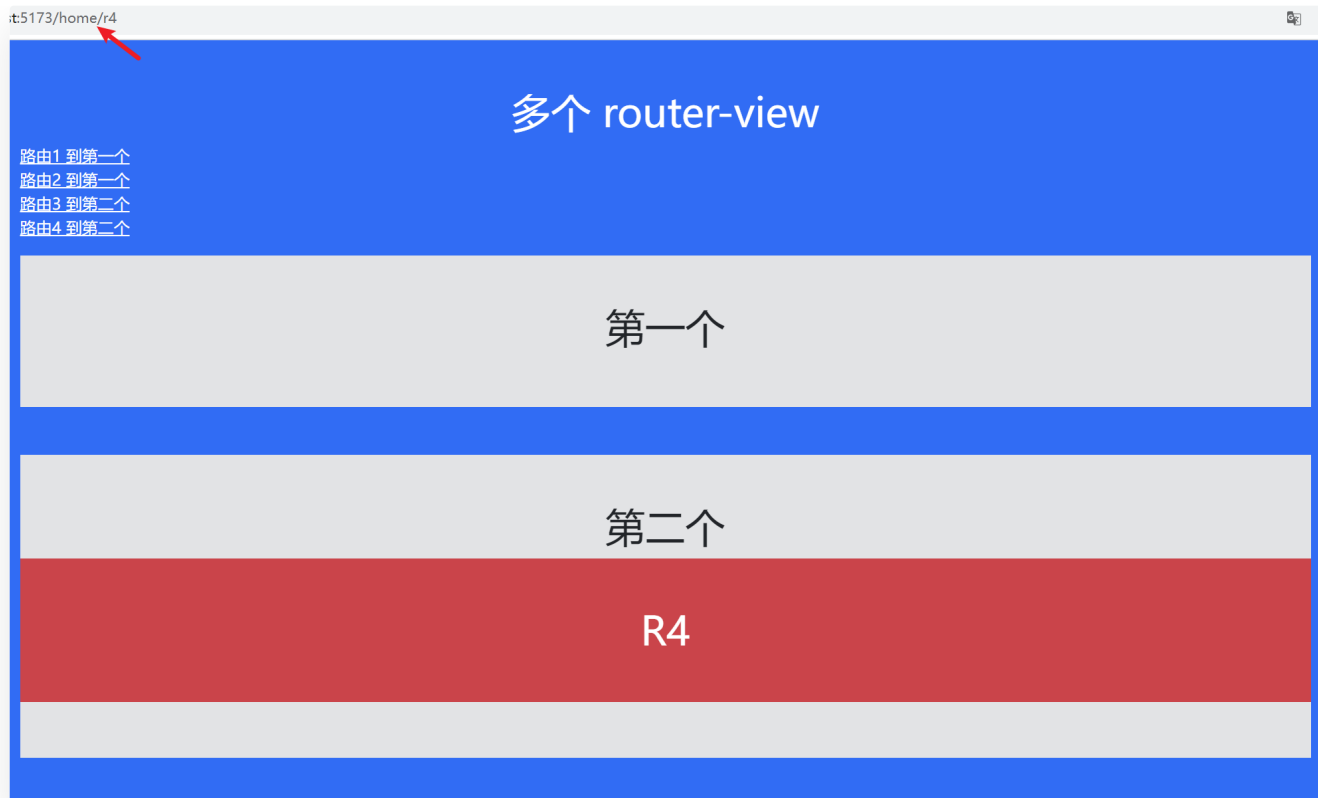
```

        name: 'r2',
        components: {
          one: () => import('@/components/R2.vue')
        }
      },
      {
        path: '/home/r3',
        name: 'r3',
        components: {
          two: () => import('@/components/R3.vue')
        }
      },
      {
        path: '/home/r4',
        name: 'r4',
        components: {
          two: () => import('@/components/R4.vue')
        }
      }
    ]
  }
})

export default router

```

- 效果



5.5 动态路由

≡ 动态路由是指：同个 URL 但携带不同参数依然路由到某个组件

- 场景：如同个商品组件根据商品ID、渲染不同商品数据

- `./src/components/About1.vue`

```
<template>
  <h1 class="py-5 text-center">About1 组件</h1>

  <ul>
    <li><router-link to="/home/about1/goods/1">商品1</router-link>
  </li>
    <li><router-link to="/home/about1/goods/2">商品2</router-link>
  </li>
    <li><router-link to="/home/about1/goods/3">商品3</router-link>
  </li>
  </ul>
</template>
```



```

    <li><router-link to="/home/about1/goods/4">商品4</router-link>
  </li>
    <li><router-link to="/home/about1/goods/5">商品5</router-link>
  </li>
</ul>

<div class="py-5 bg-secondary">
  <router-view />
</div>

</template>

```

- main.js [绑定动态路由参数]



```

import {createRouter, createWebHistory} from 'vue-router'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      redirect: '/home'
    },
    {
      path: '/*:pathMatch(.*)',
      redirect: '/home'
    },
    {
      path: '/home',
      name: 'home',
      component: () => import('@views/HomeView.vue'),
      children: [
        {
          path: '/home/about1',
          name: 'about1',
          component: () =>
import('@components/About1.vue'),
          children: [
            {
              // :动态参数
              path: '/home/about1/goods/:id',
              name: 'goods',

```

```

                                component: () =>
import('@components/Goods.vue'),
                                props: true
                                }
                                ]
                                },
                                {
                                path: '/home/about2',
                                name: 'about2',
                                component: () => import('@components/About2.vue')
                                }
                                ]
                                },
                                {
                                path: '/login',
                                name: 'login',
                                component: () => import('@views/LoginView.vue')
                                }
                                ]
                                })

export default router

```

- `./src/components/Goods.vue`



```

<template>
  <h1 class="text-center text-white">Goods : {{ id }}</h1>
</template>
<script setup>
  defineProps({id: Number})
</script>

```

- 效果 `[localhost:5173/home/about1/goods/3]`

Home

[关于1](#)[关于2](#)

About1 组件

- [商品1](#)
- [商品2](#)
- [商品3](#)
- [商品4](#)
- [商品5](#)

Goods : 3

5.6 编程式路由

≡ 声明式路由可以解决所有路由配置、Vue 还允许编程式方式控制路由

01. 机制

- 编程式路由可以解决动态绑定 `<router-link>` 麻烦的问题
- `import { useRouter } from 'vue-router'`

02. push

≡ `router.push({路由对象})` 跳转到参数路由对象

- `./src/views/LoginView.vue`

```

<template>
  <div class="container mt-5 py-5 bg-primary">
    <h1 class="text-center text-white fw-bold">Login</h1>
    <h1 class="text-center">
      <button class="btn btn-danger"
        @click="goToHome">点我跳转到首页
      </button>
    </h1>
  </div>
</template>

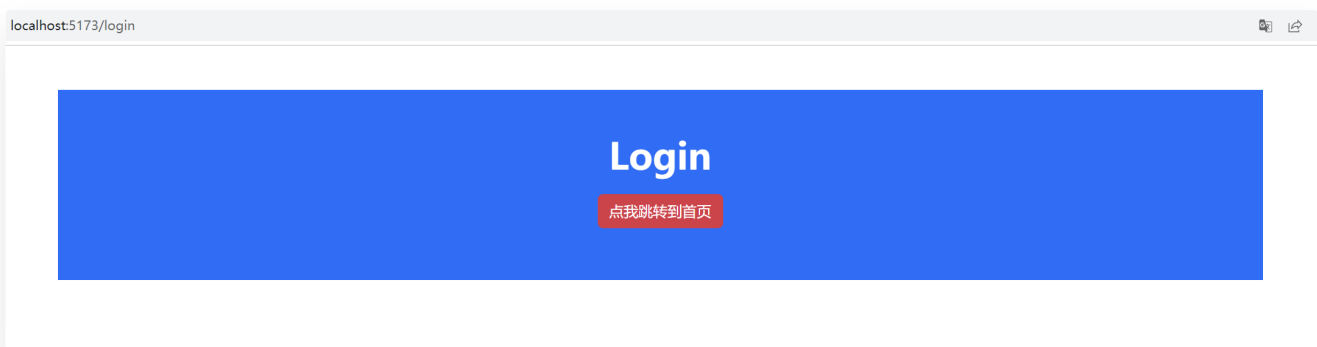
<script setup>
  import {useRouter} from 'vue-router'
  const router = useRouter()

  function goToHome() {
    // do something...
    router.push({name: 'home'})
  }

</script>

```

- 效果



03. go

≡ router.go(number) 前进或后退前面访问过的路由

- js

```
• • •  
<script setup>  
  import {useRouter} from 'vue-router'  
  const router = useRouter()  
  
  router.go(-1) 后退一次上次访问过的路由 [不是回退到父级!!!]  
  
  router.go(+1) 前进一次上次访问过的路由  
</script>
```

04. 其它

- 略

5.7 路由守卫

≡ vue router 允许在路由改变的过程中进行拦截、该拦截被称为路由守卫

- 路由守卫可用于全局路由拦截逻辑处理、用户登录拦截、权限拦截等

01. 机制说明

- 全局前置拦截: `beforeEach((to, from, next)⇒ {})` 路由还未跳转
- 全局后置拦截: `afterEach((to, from, next)⇒ {})` 路由已跳转
- 单个路由拦截: `beforeEnter:(to, from, next)⇒ {}` 配置在路由对象

参数说明

- to 表示当前去的路由对象
- from 表示当前所在页面对应的路由对象
- next({路由对象}) 表示跳转到指定参数路由、无参时表示放行

02. 拦截实例

≡ 此处演示 通过路由守卫全局设置所有页面标题

- ./src/router/index.js

```
import {createRouter, createWebHistory} from 'vue-router'

const routes = [
  {
    path: '/',
    redirect: '/home'
  },
  {
    path: '/*',
    redirect: '/home'
  },
  {
    path: '/home',
    name: 'home',
    component: () => import('@views/HomeView.vue'),
    meta: {title: '首页', isPage: true},
    children: [
      {
        path: '/home/about1',
        name: 'about1',
        component: () =>
import('@components/About1.vue'),
        children: [
          {
            path: '/home/about1/goods/:id',
```

```

        name: 'goods',
        component: () =>
import('@/components/Goods.vue'),
        props: true
    }
  ],
},
{
  path: '/home/about2',
  name: 'about2',
  component: () => import('@/components/About2.vue')
}
]
},
{
  path: '/login',
  name: 'login',
  component: () => import('@/views/LoginView.vue'),
  meta: {title: '用户登录', isPage: true},
  beforeEnter: (to, from, next) => { console.log(to, ' //
todo') }
}
]

const router = createRouter({
  history: createWebHistory(),
  routes: routes
})

router.beforeEach((to, from, next) => {

  if (to?.meta?.title && to?.meta?.isPage) {
    document.title = to.meta.title + ' - ' + 'XX系统'
  }

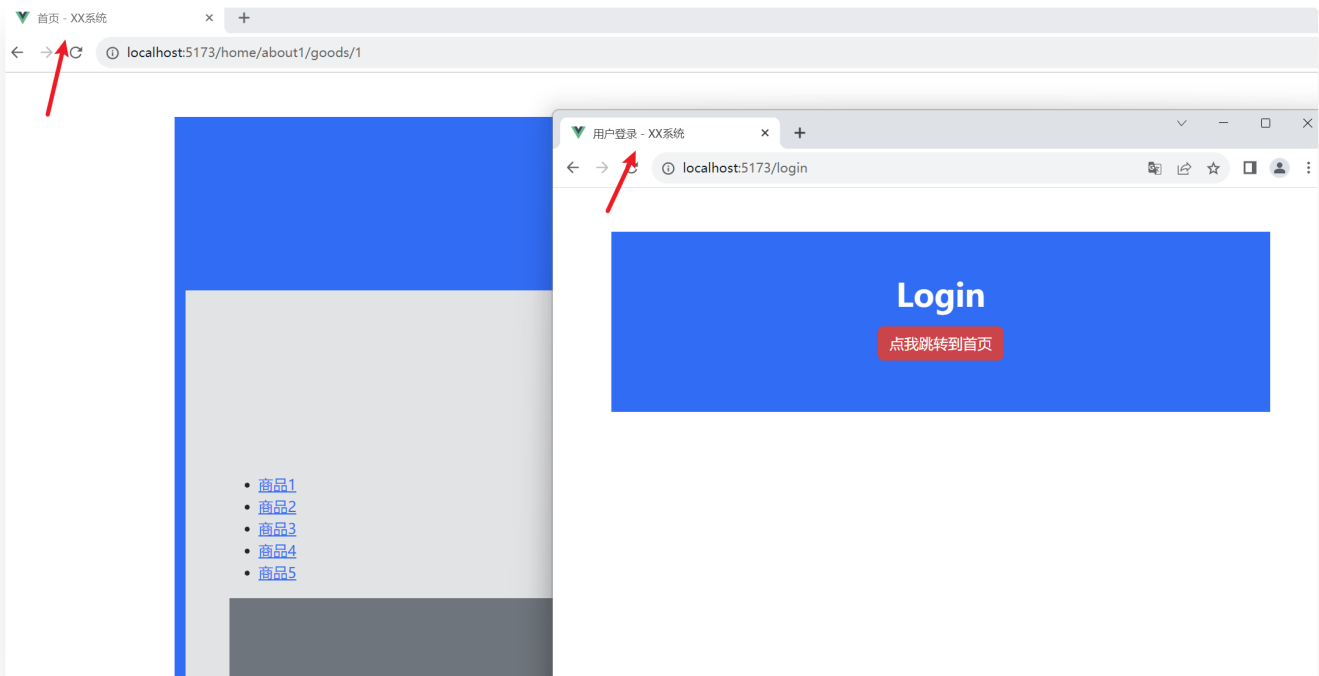
  // 放行
  next()

  // 跳转到 home
  // next({name: 'home'})

})
export default router

```

- 效果



06. Pinia

≡ 官网: <https://pinia.vuejs.org/zh/>



6.1 状态管理概念

- 将全局数据或多个组件 **共用数据及状态修改** 定义在 pinia 中、即为状态管理
- pinia 是针对 vue3 的状态管理工具、vuex 对应 vue2 版本

6.2 安装

≡ pinia 并未集成在 vue 中、使用前还需进行安装和挂载

- npm



```
npm install pinia
```

- main.js [挂载 pinia]



```
import {createApp} from 'vue'
import App from './App.vue'

import {createPinia} from 'pinia'

const pinia = createPinia()

const app = createApp(App)

app.use(pinia)
app.mount('#app')
```

6.3 模块定义

≡ 状态管理模块约定于 ./src/stores/xxx.js 目录、一个文件对应一个状态管理模块

- defineStore('状态模块ID', () ⇒ { 返回状态、状态更新函数 })
- ./src/stores/user.js [用户状态管理]



```
import {defineStore} from 'pinia'
import {ref} from 'vue'

const useUserStore = defineStore('user', () ⇒ {
```

```

const username = ref('lucas')
const age = ref(20)

const user = ref({
  name: 'jerry',
  gender: 'male'
})

return {username, age, user}
})

export default useUserStore

```

6.4 状态使用

≡ 在 pinia 中完成状态定义、状态维护后、组件中直接导入该模板使用即可

- storeToRefs(userStore) 解构可保持响应式、否则将会丢失
- app.vue



```

<script setup>
  import {v4 as uuid} from 'uuid'
  import useUserStore from '@/stores/user'
  import {storeToRefs} from 'pinia'

  const userStore = useUserStore()
  const {username, age, user} = storeToRefs(userStore)

  setInterval(() => {
    age.value = Math.random()
    username.value = uuid()
    user.value = {
      name: uuid(),
      gender: Math.random() > 0.5 ? 'male' : 'female'
    }
  })

```

```
    }, 2000)

</script>

<template>
  <h1>username : {{ username }}</h1>
  <h1>age : {{ age }}</h1>
  <h1>user : {{ user }}</h1>
</template>
```

6.5 持久化

≡ 持久化允许将 pinia 中的状态存储在 session/localStorage 对象上

- pinia 持久化需安装插件 pinia-plugin-persistedstate 来支持
- npm

```
● ● ●
npm install pinia-plugin-persistedstate
```

- main.js

```
● ● ●
import {createApp} from 'vue'
import App from './App.vue'

import piniaPluginPersistedState from 'pinia-plugin-persistedstate'

import {createPinia} from 'pinia'
```

```
const pinia = createPinia()

// 将 piniaPluginPersistedState 插件挂载到 pinia 上
pinia.use(piniaPluginPersistedState)

const app = createApp(App)
app.use(pinia)

app.mount('#app')
```

- user.js [配置持久化 调用一次 updateState 将会自动触发存储]

```
import {defineStore} from 'pinia'
import {reactive} from 'vue'

const useUserStore = defineStore('user', () => {

  const state = reactive({
    username: 'none', isLogin: false
  })

  function updateState(username, isLogin) {
    state.username = username
    state.isLogin = isLogin
  }

  return {state, updateState}

}, {persist: {storage: sessionStorage}})

export default useUserStore
```

07. 组合式api

≡ 本课件全以 `<script setup>` 方式演示、该方式被称为组合式api

- `<script setup>` 是组合式 api 的 setup 语法糖
- 注明 setup script 元素中可直接定义变量、函数、使用生命周期钩子等

- app.vue

```
● ● ●  
  
<template>  
  <h1>username: {{ username }}</h1>  
</template>  
  
<script setup>  
  import { ref } from 'vue'  
  const username = ref('天气预报')  
</script>
```

≡ 非 setup 语法糖则需将 变量、函数 定义在 setup 函数中并返回

- app.vue [非 setup 语法糖]

```
● ● ●  
  
<template>  
  <h1>username: {{ username }}</h1>  
  <h1>age : {{ age }}</h1>  
</template>  
  
<script>  
  import { ref } from 'vue'  
  
  export default {
```

```
    setup() {  
      const username = ref('天气预报')  
      const age = 18  
      return {username, age}  
    }  
  }  
</script>
```

08. 开源组件

≡ 高质量组件收集中...

8.1 Paginate

≡ vue-awesome-paginate 轻量级的分页插件

- npm



```
npm install vue-awesome-paginate
```

- app.vue



```
<script setup>
  import {VueAwesomePaginate} from 'vue-awesome-paginate'
  import {ref} from 'vue'

  const total = ref(1000)
  const page = ref(1)
  const size = 20
  const showMaxButtons = 5

  function onClickHandler(page) {
    console.log(`page = ${page}`)
  }
</script>

<template>
  <vue-awesome-paginate
    :total-items="total"
    :items-per-page="size"
    :max-pages-shown="showMaxButtons"
    v-model="page"
    :on-click="onClickHandler"
    prev-button-content=" <"
    next-button-content="> "
  />
</template>
```

- vue-awesome-paginate 分页样式



```
<style lang="scss">

// 分页容器
.pagination-container {
  list-style: none;
  display: flex;
  gap: 8px;
  padding: 0;
  width: fit-content;
  margin: 50px auto;
  transition: all .3s;

  // 分页按钮
```



```
.paginate-buttons {
  all: unset;
  width: 36px;
  height: 36px;
  line-height: 36px;
  text-align: center;
  font-family: "system-ui";
  font-size: 16px;
  font-weight: bold;
  color: #0d6efd;

  // 分页按钮 hover、选中
  &.active-page,
  &:hover {
    color: #fff;
    background-color: #0d6efd;
    cursor: pointer;
  }

  // 头尾按钮 及 hover
  &.back-button,
  &.next-button {
    font-size: 15px;
    color: #0d6efd;
    background-color: #f8f9fa;

    &:hover {
      background-color: #eee;
    }
  }
}

}

// 小屏缩小
@media (max-width: 768px) {
  .pagination-container {
    gap: 5px;

    .paginate-buttons {
      width: 30px;
      height: 30px;
      line-height: 30px;
    }
  }
}

</style>
```

- 效果

< 1 2 3 4 5 ... 50 >

8.2 VeeValidate

≡ <https://vee-validate.logaretm.com/v4/> 轻量的表单校验

- npm

```
npm install vee-validate
npm install yup

npm install bootstrap sass
```

- app.vue

```
<template>
  <div class="container">
    <div class="row">
      <div class="col-lg-7 offset-lg-2 px-3 py-5 mt-5 bg-light">
        <form action="">
          <div class="mb-3">
            <div class="input-group input-group-lg">
              <span class="input-group-text rounded-0 text-primary">
                <i class="bi bi-person-fill"></i>
              </span>
              <input type="text" class="form-control rounded-0">
            </div>
          </div>
        </form>
      </div>
    </div>
  </div>
</template>
```

```

        <p class="ms-2 mt-2 text-muted">账号不能包含中文等特殊符号、必须
        须以字母开头。</p>
        <p class="text-danger ms-2">账号格式不正确、请检查后重新输入。
    </p>
    </div>

    <div class="mb-3">
        <div class="input-group input-group-lg">
            <span class="input-group-text rounded-0 text-primary">
                <i class="bi bi-lock-fill"></i>
            </span>
            <input type="text" class="form-control rounded-0">
        </div>
        <p class="ms-2 mt-2 text-muted">密码不能包含中文等特殊符号、必
        须是字母、数字、_下划线</p>
        <p class="text-danger ms-2">密码格式不正确、请检查后重新输入。
    </p>
    </div>

    <div class="mb-3">
        <div class="input-group input-group-lg">
            <input type="text" class="form-control rounded-0">
            
        </div>
        <p class="text-danger ms-2">验证码格式不正确、请检查后重新输入。
    </p>
    </div>

    <div>
        <button class="btn btn-primary w-100 btn-lg rounded-0 mt-
3">提 交</button>
    </div>

    </form>
</div>
</div>
</div>
</template>

```

- 效果

vee-validate



账号不能包含中文等特殊符号、必须以字母开头。

账号格式不正确、请检查后重新输入。



密码不能包含中文等特殊符号、必须是字母、数字、_下划线

密码格式不正确、请检查后重新输入。



验证码格式不正确、请检查后重新输入。

提交

≡ 校验绑定

- setup

```

<script setup>

import {useForm} from 'vee-validate'
import * as yup from 'yup'

const {errors, handleSubmit, defineInputBinds} = useForm({
  validationSchema: yup.object({
    username: yup.string().required('请输入账号。').min(6, '账号至少6位 my son。'),
    password: yup.string().required('请输入密码。').matches(/^[a-z0-9]{8,}$/ , '密码不正确 my son。'),
    code: yup.string().required('请输入验证码。').length(4, '验证码必须是4位 my son。')
  })
})

const username = defineInputBinds('username')
const password = defineInputBinds('password')
```

```

const code = defineInputBinds('code')

const onSubmit = handleSubmit(function ({username, password, code}) {

  console.log(username, password, code)

}, function ({errors}) {
  const firstError = Object.keys(errors)[0];
  document.querySelector(`[name="${firstError}"]`).focus()
})

</script>

```

- app.vue



```

<template>
  <div class="container">
    <div class="row">
      <div class="col-lg-7 offset-lg-2 px-3 py-5 mt-5 bg-light">

        <h1 class="mb-3 text-center fw-bold">vee-validate</h1>

        <form action="" @submit="onSubmit">
          <div class="mb-3">
            <div class="input-group input-group-lg">
              <span class="input-group-text rounded-0 text-primary">
                <i class="bi bi-person-fill"></i>
              </span>

              <input type="text"
                class="form-control rounded-0"
                v-bind="username"
                name="username">

            </div>
            <p class="ms-2 mt-2 text-muted">账号不能包含中文等特殊符号、必须
            须以字母开头。 </p>
            <p class="text-danger ms-2">{{errors.username}}</p>
          </div>

          <div class="mb-3">
            <div class="input-group input-group-lg">
              <span class="input-group-text rounded-0 text-primary">
                <i class="bi bi-lock-fill"></i>
              </span>

```

```

    </span>

    <input type="text"
      class="form-control rounded-0"
      v-bind="password"
      name="password">

  </div>
  <p class="ms-2 mt-2 text-muted">密码不能包含中文等特殊符号、必
  须是字母、数字、_下划线</p>
  <p class="text-danger ms-2">{{errors.password}}</p>
</div>

<div class="mb-3">
  <div class="input-group input-group-lg">

    <input type="text"
      class="form-control rounded-0"
      v-bind="code"
      name="code">

  </div>
  <p class="text-danger ms-2 mt-3">{{errors.code}}</p>
</div>

<div>
  <button class="btn btn-primary w-100 btn-lg rounded-0 mt-
3">提 交</button>
</div>

</form>
</div>
</div>
</div>
</template>

```

- 效果

vee-validate



账号不能包含中文等特殊符号、必须以字母开头。



密码不能包含中文等特殊符号、必须是字母、数字、_下划线

密码格式不正确。



请输入验证码。

提交

09. 友情链接

≡ 下为常用技术栈站点或其中文网链接

- vue : <https://cn.vuejs.org/>
- vue-router : <https://router.vuejs.org/zh/>
- pinia : <https://pinia.vuejs.org/zh/>
- bootstrap : <https://v5.bootcss.com/>
- tailwindcss : <https://www.tailwindcss.cn/>
- npm : <https://npm.p2hp.com/>
- axios : <https://www.axios-http.cn/docs/intro>
- react : <https://react.docschina.org/>
- sass : <https://www.sass.hk/>

- **typescript** : <https://ts.nodejs.cn/>